CS4269

Wang Xiyu A0282500R

Contents

1 Syntax of Propositional Logics

Definition 1.1. *DSyntax decides what are allowed to be written*

1.1 Well-formed Formula (WFF)

A formula ψ is obtained by composing $p \in P$ with $c \in C$, where P is the set of propositions and C is the set of connectors. here we use any set of complete logic, like $C = \{\neg, \vee\}$. This is not a percise definition as counter examples that do not form a valid formula can be formulated based on this, for example $\neg \vee p$ means nothing. To formalize the definition we may attempt to define a language with the following grammar:

$$\Sigma = P \cup C \cup \{(,)\}$$

$$FORM : \{\psi = p \in P \mid \neg\psi \mid \psi \vee \psi \mid (\psi)\}$$

However this is a recursive definition as the definition of the a formula ψ involves itself. We need to find a declarative way to define this recursive/inductive construction.

Definition 1.2. *FORM is **the smallest** set of strings over Σ where*

1. $P \subseteq FORM$
2. $\forall \psi \in FORM, \neg\psi \in FORM$
3. $\forall \psi_1, \psi_2 \in FORM, \psi_1 \vee \psi_2 \in FORM$

We can see as a construction rule, we are admitting strings into the set of formula, but we need to somewhat impose restriction on what is not acceptable.

1.2 Closed Set

Let's remove the attributive **the smallest** first and define another set that contains all the set that satisfy these rules listed above:

Definition 1.3. *Closed is a set of strings over Σ where*

1. $P \subseteq Closed$
2. $\forall \psi \in Closed, \neg\psi \in Closed$
3. $\forall \psi_1, \psi_2 \in Closed, \psi_1 \vee \psi_2 \in Closed$

1.2.1 All Closed Set

And we define a set ACS (All-closed sets). We can define FORM via the following constriction:

$$FORM = \bigcap ACS$$

Suppose $BCS = \emptyset$ and from definition

$$U_0 = \bigcap BCS = \{w \in \Sigma^* \mid \forall S \in BCS, w \in S\}$$

However, $BCS = \emptyset$, The statement $\forall S \in BCS, w \in S$ is vacuously true $\forall w \in \Sigma^*$, which means that

$$U_0 = \Sigma^* = \bigcap BCS$$

A contradiction. Therefore we have proved that ACS cannot be empty, thus $\forall P, \exists S$ such that S is a closed set.

1.3 FORM is the minimum of the set of Closed

First let's prove that FORM is closed. It seems to be trivial that the intersection of closed sets is closed but formal proof is still needed. We denote a set from the intersection construction as T_0.

Lemma 1.4. *T_0 is closed*

We are proving that the set T_0 we get from the intersection construction still satisfy the rules that define what is closed.

Proof. From rule 1,

$$\begin{aligned}
&\forall S \in ACS, P \subseteq S \\
&\quad \to \forall p \in P, \forall S \in ACS, p \in S \\
&\quad \to \forall p \in P, p \in \bigcap ACS \\
&\quad \to P \in T_0
\end{aligned}$$

From rule 2,

$$\begin{aligned}
&\forall S \in ACS, T_0 \subseteq S \\
&\text{let } w \in T_0, \forall S \in ACS, w \in S, \neg w \in S \\
&\quad \to \forall w \in T_0, \neg w \in \bigcap ACS \\
&\quad \to \forall w \in T_0, \neg w \in T_0
\end{aligned}$$

From rule 3,

$$\begin{aligned}
&\forall S \in ACS, T_0 \subseteq S \\
&\text{let } w_1, w_2 \in T_0, \forall S \in ACS, w_1 \vee w_2 \in S \\
&\quad \to \forall w_1, w_2 \in T_0, w_1 \vee w_2 \in \bigcap ACS \\
&\quad \to \forall w_1, w_2 \in T_0, w_1 \vee w_2 \in T_0
\end{aligned}$$

□

Lemma 1.5. *T_0 is **a** smallest Closed set*

Proof. The proof of minimality is trivial. $T_0 = \bigcap ACS \to \forall S \in ACS, T_0 \subseteq S$ □

Lemma 1.6. *T_0 is **the** smallest Closed set (unique). In other word,*

$$\forall T_0' \in \{T \mid \forall S \in ACS, T \subseteq S\}, T_0' = T_0$$

Now we can use the generative rule to define formula, which is called **Backus-Naur form**. Note that a context-free grammar is a type of formal language. Backus Naur form is a specification language for this type of grammar. It is used to describe language syntax.

1.4 Subformula

Definition 1.7. *ϕ_1 is a subformula of ϕ_2, denoted by*

$$\phi_1 \preccurlyeq \phi_2$$

as a relation is

- *reflexive:* $\forall \phi \in FORM, \phi \preccurlyeq \phi$
- *transitive:* $\forall \phi_1, \phi_2, \phi_3 \in FORM, (\phi_1 \preccurlyeq \phi_2) \wedge (\phi_2 \preccurlyeq \phi_3) \to \phi_1 \preccurlyeq \phi_3$

Inductive definition:

$$\forall \phi \in FORM, \phi \preccurlyeq (\neg \phi)$$
$$\forall \phi_1, \phi_2 \in FORM, \phi_1 \preccurlyeq (\phi_1 \vee \phi_2)$$
$$\phi_2 \preccurlyeq (\phi_1 \vee \phi_2)$$

We might intuitively think that subformula can be defined using the notion of substring, since we have been considering formula as concatenation of proposition and connector symbols. However based on out inductive definition, we can come up with the following counter example to a definition using substring: A formula ϕ_1 is a subformula of ϕ_2 if and only if ϕ_1 is a well-formed formula, and is a substring to ϕ_2

$$p \vee q \not\preccurlyeq \neg p \vee q$$

What if we tighten the syntax by strictly enforce the use of parenthese whenver we can

2 Sematics of Propositional Logics

Definition 2.1. *Sematics describe meanings.*

2.1 Truth Assignment/Proposition evaluation

A truth assignment or a valuation of propositions is a function that maps from the set of propositions to true and false.

$$v : P \to \{T, F\}$$
$$v \in [P \to \{T, F\}]$$

Example 2.2. *Consider* $v = \{p_1 \mapsto T, p_2 \mapsto F\}$*, we can derive*

$$v(p_1) = T$$
$$v(p_2) = F$$
$$v(\neg p_2) = T$$
$$v(p_1 \vee p_2) = T \ \textit{etc.}$$

2.2 Entail/Model

We denote entailment of an evaluation function to formula as

$$v \models \psi$$

We also indeuctively define the following rule

$$\forall p \in P, v \models p \iff v(p)$$
$$v \models \psi \iff v \not\models \neg\psi$$
$$v \models \psi_1 \vee \psi_2 \iff (v \models \psi_1) \vee (v \models \psi_2)$$

2.2.1 Valuation and Model

A valuation v is a model of a formula ϕ, if and only if $v(\phi) = T$.

- $\forall \phi$ over $P, \exists 2^{|P|}$ valuations.
- A formula can have many or one or none models

2.2.2 $\models$ as a relation

Definition 2.3. $\models \ \subseteq \{2^{|P|} \times FORM\}$ *is a binary relation, between truth assignment to formula. We define as follows:*

- $\forall p \in P, v \models p \iff v(p) = 1$

- $v \not\models p$ *otherwise.*

Consider a valuation v as a subset of P such that the propositions in v denoted by $v(p)$ is assigned with true. $v \in \mathscr{P}(P)$

ϕ	ψ	$\neg\phi$	$(\phi \wedge \psi)$	$(\phi \vee \psi)$	$(\phi \to \psi)$	$(\phi \leftrightarrow \psi)$
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

Definition 2.4.

$$p(v) = v(p)$$

A proposition can be viewed as a circuit is jsut a wire, thus if a proposition is assigned with true by a valuation, the valuation valuates this proposion to be true

Lemma 2.5. *Let* $\phi \in FORM, v \in \mathscr{P}(P), v \models \phi \iff \phi(v) = 1$

Proof. Base case: $\forall \phi = p \in P, v \models \phi \iff v(p) = 1 \iff p(v) = 1$
Inductive case:

1.

□

2.2.3 Logical Equivalence

Definition 2.6. *Syntax definition of logical equivalence is as follows:*

$$\phi_1 \equiv \phi_2 \iff \forall v : P \mapsto \{T, F\}(v \models \phi_1), v \models \phi_2$$

Example 2.7.

$$p \vee (\neg p) \equiv q \vee (\neg q)$$

$$\forall v \models (p \vee (\neg p)),$$
$$v \models p \ or \ v \models (\neg p)$$
$$v \models p \ or \ v \not\models p$$
$$\forall v : P \mapsto \{T, F\},$$

TODO

2.3 Satisfiability of Formula

Definition 2.8. *Let* $\phi \in FORM, \phi$ *is satisfiable*

$$\iff \exists v : P \mapsto \{T, F\}, v \models \phi$$

2.3.1 Valid formula(tautology)

Definition 2.9. *Tautology:* ϕ *is a tautology* $\iff \forall v, v \models \phi$

Example 2.10.

$$\phi = \neg(p \vee (\neg p))$$

From example 2.4, we have

$$\begin{aligned}&\forall v, v \models (p \vee (\neg p))\\ &\rightarrow \forall v, v \not\models \neg(p \vee (\neg p))\end{aligned}$$

Thus ϕ is NOT satisfiable.

2.4 Satisfiability of a set of formula

Set Satisfication

Definition 2.11. *Let $\Gamma \subseteq FORM$,*

$$v \models \Gamma \iff \forall \phi \in \Gamma, v \models \phi$$

Set Satisfiability

Definition 2.12. *Let $\Gamma \subseteq FORM$, Γ is satisfiable*

$$\iff \exists v, \forall \phi \in \Gamma, v \models \phi$$

Example 2.13. *$\Gamma = \emptyset$, Γ is satisfiable.*

Proof. Consider $v : P \mapsto \{T, F\}, v(p) = T \forall p \in P$, $\forall \phi \in \Gamma, v \models \phi$ is vacuously true. □

Example 2.14. *$\Gamma = FORM$, Γ is NOT satisfiable.*

Proof. Suppose otherwise,

$$\begin{aligned}&\exists v, \forall \phi \in \Gamma, v \models \phi\\ &\forall \phi \in \Gamma, (\neg\phi) \in \Gamma\\ &v \models (\neg\phi)\end{aligned}$$

A contradiction. □

2.4.1 Relevance

Definition 2.15. *Define set $OCCUR(\phi) \subseteq P$, which contains propositions appear in ϕ. Inductively defined as*

$$\begin{aligned}&OCCUR(\phi) = \{p\} \iff \phi \equiv p, p \in P\\ &OCCUR(\neg\phi) = OCCUR(\phi)\\ &OCCUR(\phi_1 \vee \phi_2) = OCCUR(\phi_1) \cup OCCUR(\phi_2)\end{aligned}$$

Lemma 2.16. *Let $\phi \in FORM$, let v_1, v_2 be valuations over P.*

$$\forall p \in OCCUR(\phi), v_1(p) = v_2(p) \rightarrow (v_1 \models \phi \rightarrow v_2 \models \phi)$$

v_1, v_2 can be different valuations over $p' \notin OCCUR(\phi)$

Proof. Let $p \in P$, □

2.4.2 Logical consequence

Semantic logical consequence:

Definition 2.17.

$$\Gamma \models \phi \iff \forall v \models \Gamma, v \models \phi$$

Syntactic logical consequence: (logical deriability)

Definition 2.18. *A formula ϕ is a syntactic consequence of Γ if there exists a formal proof (a finite sequence of formulas) ending in ϕ, where each step is either a premise from Γ, an axiom, or follows from previous steps via an inference rule (like Modus Ponens).*

Material Implication ($\rightarrow$) vs. Logical Consequence ($\models$)

- Material implication is a connector within a formula, $p \rightarrow q \equiv \neg(p \vee q)$, and can therefore be eval to true or false in a single model.
- Logical consequence is a relation between formulas (meta-logic), it is a statement about all models. $\{p, p \rightarrow q\} \models q$

2.5 Modelling Computations as Propositional Logic

Computation problems can be considered as membership check of an encoded problem instance to a language.

$$L = \{p \mid \langle M, p \rangle\}$$

Here instead of arbituary encoding, we consider problem encoding with propositional logics only, and solve with SAT solvers by deciding whether the set of formula we form from any instance of the problem is satisfiable. That is

$$Encode : p \mapsto FORM, \forall p, \exists v \models Encode(p) \iff p \in L$$

The rules of such modelling include:

- Propositional logics only
- The final expression must one well formed fomula
- for problem of size n, the size of the formula ϕ must be in $O(poly(n))$

Note that there can be different definition on the size of the

- problem: in unary, binary etc;
- Formula: Whether by the number of nodes in the syntax tree or length of the string Anything else?

Example 2.19. *Graph coloring problem: given a undirected graph $G = (V, E)$, whether there exists a k coloring of all vertices such that no neighbouring vertices have the same color.*
Consider the following construction of a proposition set P:

$$P = \{p_{v,i} \mid v \in V, i \in [1, k]\}$$

The valuation of proposition $p_{v,i}$ conceptually represents vertex v is assigned color i.
We can formulate constriants as well formed formula, and use conjunction to connect the formula to model satisfaction of multiple constraints.

Corollary 2.20. $\forall \phi_1, \phi_2 \in FORM, (\phi_1 \wedge \pi_2) \in FORM$

First let us list down the constraints of this problems, including implicit ones in plain words

1. *Neighbouring vertices have different colors.*
2. *All vertices as assigned with exactly one color.*

1. ϕ_{nbr}

First lets formula the constraint between any 2 vertices,

$$\forall (\alpha, \beta) \in E, c(v_\alpha) \neq c(v_\beta)$$

And from our proposition set, $c(v_\gamma) = i \iff \exists v, v \models p_{\gamma,i}$. As such we have

$$\phi_{\alpha,\beta,i} = \neg(p_{\alpha,i} \wedge p_{\beta,i})$$

This models vertex α and vertex β are not colored with color i at the same time. For every color this needs to hold, thus we have

$$\phi_{nbr(\alpha,\beta)} = \bigwedge_{i \in [1,k]} \phi_{\alpha,\beta,i}$$

For every pair of neighbouring vertices this also needs to hold. Therefore we have

$$\phi_{nbr} = \bigwedge_{(\alpha,\beta) \in E} \phi_{nbr(\alpha,\beta)}$$

2. $\phi_{exactly-1}$

To model exactly 1 coloring on each vertex, we can formulate $\phi_{\geq 1}$ *and* $\phi_{\leq 1}$ *first and use conjunction to connect them. Given our propositions, to make sure vertex* v *is at least assign a color* ($\phi_{v,\geq 1}$),

$$\phi_{v,\geq 1} = \bigvee_{i \in [1,k]} p_{v,i}$$

This needs to hold for every vertex, thus we have

$$\phi_{\geq 1} = \bigwedge_{v \in V} \phi_{v,\geq 1}$$

Then we need to model at most 1, (or less than 2) coloring on each vertex. Vertex v *is not assignment with both color* i *and* j *can be formulated as*

$$\phi_{v,i,j,\leq 1} = \neg(p_{v,i} \wedge p_{v,j})$$

This needs to hold for every distinct pair of colors, thus

$$\phi_{v,\leq 1} = \bigwedge_{i \neq j \in [1,k]} \phi_{v,i,j,\leq 1}$$

For all vertices

$$\phi_{\leq 1} = \bigwedge_{v \in V} \phi_{v,\leq 1}$$

Finally we have

$$\phi_{exactly-1} = \phi_{\leq 1} \wedge \phi_{\geq 1}$$

And the overall formula for an instance of $G = (V, E)$ *becomes*

$$\phi = \phi_{nbr} \wedge \phi_{exactly-1}$$

Similar to reduction, we need also now to show that

$$\exists v : P \mapsto \{T, F\}, v \models \phi \iff \langle G = (V, E), k \rangle \in L_{color}$$

TODO

Introduce propositional variables that encode atomic choices, then add constraints that enforce the combinatorial structure of the problem.

1. Hamiltonian Path

Instance. Graph $G = (V, E)$ with $|V| = n$.

Variables. For every vertex $v \in V$ and position $i \in \{1, \ldots, n\}$:

$$X_{v,i} \quad \text{“vertex } v \text{ is at position } i \text{ in the path.”}$$

Constraints.

1. **Each position has exactly one vertex:**

$$\bigvee_{v \in V} X_{v,i} \quad \text{and pairwise exclusion.}$$

2. **Each vertex appears exactly once:**

$$\bigvee_{i=1}^{n} X_{v,i} \quad \text{and pairwise exclusion.}$$

3. **Adjacency constraint:** If $(u, v) \notin E$, then for all i:

$$\neg X_{u,i} \vee \neg X_{v,i+1}.$$

Satisfiable iff G has a Hamiltonian path.

2. Clique of Size k

Instance. Graph $G = (V, E)$.

Variables. For each vertex $v \in V$:

$$X_v \quad \text{“vertex } v \text{ is selected.”}$$

Constraints.

1. **At least k vertices selected:** Encode using a cardinality constraint.
2. **Selected vertices form a clique:** If $(u, v) \notin E$, then
$$\neg X_u \vee \neg X_v.$$

Satisfiable iff there exists a clique of size $\geq k$.

3. 3-Dimensional Matching

Instance. Sets X, Y, Z and triples $T \subseteq X \times Y \times Z$.

Variables. For each triple $t \in T$:

$$X_t \quad \text{“triple } t \text{ is chosen.”}$$

Constraints.

1. **No element used twice:**
If triples t, t' share an element, then
$$\neg X_t \vee \neg X_{t'}.$$
2. **Select exactly k triples:** Encode cardinality constraint.

Satisfiable iff there is a matching of size k.

4. Subset Sum

Instance. Numbers $a_1, \ldots, a_n$, target B.

Variables.

$$X_i \quad \text{“}a_i \text{ is selected.”}$$

Constraints. We encode:

$$\sum_{i=1}^{n} a_i X_i = B.$$

This is reduced to propositional logic by building a binary addition circuit:

- Introduce variables for each bit of partial sums.
- Encode full-adder constraints using propositional clauses.

Satisfiable iff there exists a subset summing to B.

5. Exact Cover

Instance. Universe U, collection $\mathcal{S} = \{S_1, \ldots, S_m\}$.

Variables.

$$X_i \quad \text{“set } S_i \text{ is selected.”}$$

Constraints. For every element $u \in U$:

1. **At least one covering set:**
$$\bigvee_{u \in S_i} X_i$$
2. **At most one covering set:** If $u \in S_i \cap S_j$ and $i \neq j$:
$$\neg X_i \vee \neg X_j.$$

Satisfiable iff an exact cover exists.

6. Scheduling with Precedence Constraints

Instance. Tasks $T_1, \ldots, T_n$, time slots $1, \ldots, m$.

Variables.

$X_{i,t}$ "task i scheduled at time t."

Constraints.

1. Each task scheduled exactly once.
2. At most one task per time slot.
3. If T_i must precede T_j:
$$X_{i,t} \to \bigvee_{t' > t} X_{j,t'}.$$

2.6 Encoding a Full Adder in Pure Propositional Logic

A full adder takes three input bits:

$$x,\ y,\ c_{in}$$

and produces two outputs:

$$s \quad \text{(sum bit)}, \qquad c_{out} \quad \text{(carry bit)}$$

Semantically:

$$s = x \oplus y \oplus c_{in}$$
$$c_{out} = (x \wedge y) \vee (x \wedge c_{in}) \vee (y \wedge c_{in})$$

We now encode this purely in propositional logic using only $\wedge, \vee, \neg$ (or $\to, \bot$ if preferred).

1. Encoding the Sum Bit

The sum is true iff an odd number of inputs are true.

Truth table characterization:

$$s \leftrightarrow (x \wedge \neg y \wedge \neg c_{in}) \vee (\neg x \wedge y \wedge \neg c_{in}) \vee (\neg x \wedge \neg y \wedge c_{in}) \vee (x \wedge y \wedge c_{in})$$

Thus we encode:

$$(s \to \Phi_s) \ \wedge \ (\Phi_s \to s)$$

where Φ_s is the disjunction above.

2. Encoding the Carry Bit

Carry is true iff at least two inputs are true:

$$c_{out} \leftrightarrow (x \wedge y) \vee (x \wedge c_{in}) \vee (y \wedge c_{in})$$

Again encode as:

$$(c_{out} \rightarrow \Phi_c) \ \wedge \ (\Phi_c \rightarrow c_{out})$$

3. CNF Encoding (SAT-friendly)

To encode for SAT, we introduce clauses.

Carry Constraints

$$\neg c_{out} \vee x \vee y$$
$$\neg c_{out} \vee x \vee c_{in}$$
$$\neg c_{out} \vee y \vee c_{in}$$

$$\neg x \vee \neg y \vee c_{out}$$
$$\neg x \vee \neg c_{in} \vee c_{out}$$
$$\neg y \vee \neg c_{in} \vee c_{out}$$

Sum Constraints (XOR Form) We can encode $s = x \oplus y \oplus c_{in}$ using clauses:

$$(x \vee y \vee c_{in} \vee \neg s)$$
$$(\neg x \vee \neg y \vee c_{in} \vee \neg s)$$
$$(\neg x \vee y \vee \neg c_{in} \vee \neg s)$$
$$(x \vee \neg y \vee \neg c_{in} \vee \neg s)$$

$$(\neg x \vee \neg y \vee \neg c_{in} \vee s)$$
$$(x \vee y \vee \neg c_{in} \vee s)$$
$$(x \vee \neg y \vee c_{in} \vee s)$$
$$(\neg x \vee y \vee c_{in} \vee s)$$

These clauses exactly encode the XOR behavior.

4. Circuit-Style Encoding (Cleaner)

Instead of expanding XOR directly, introduce auxiliary variables:

$$t = x \oplus y$$
$$s = t \oplus c_{in}$$

Each XOR can be encoded with 4 clauses:

$$(t \leftrightarrow x \oplus y)$$

which in CNF becomes:

$$(\neg x \vee \neg y \vee \neg t) \wedge (x \vee y \vee \neg t) \wedge (x \vee \neg y \vee t) \wedge (\neg x \vee y \vee t)$$

This keeps formula size linear.

5. Why This Matters

Full adders are the atomic building blocks of:

- Binary addition circuits
- Subset Sum encodings
- Integer programming to SAT reductions
- Cook–Levin tableau arithmetic

Any arithmetic constraint can be reduced to propositional logic by composing full adders.

Conceptual Insight

Arithmetic $\longrightarrow$ Circuits $\longrightarrow$ Propositional Logic

This translation is what makes SAT complete for NP: even numeric computation can be flattened into Boolean structure.

General Pattern

Almost all NP problems can be modelled by:

1. Encoding choices with propositional variables.
2. Enforcing structural constraints with local clauses.
3. Encoding counting via cardinality circuits if needed.

This modelling perspective is precisely what makes SAT solvers universal engines for combinatorial search.

3 Compactness Theorem for Propositional Logics

Definition 3.1. *Axiom of Choice: formally*

$$\forall X, \emptyset \in X \vee \exists f, dom(f) = X \wedge \forall A \in X, f(A) \in A$$

A choice function (or selection) f defined on a collection of nonempty sets X, such that for every set A in the collection X, $f(A)$ is an element in A. That is, Axiom—For any set X of nonempty sets, there exists a choice function f that is defined on X and maps each set of X to an element of that set.

Theorem 3.2. *If P is finite, $FORM$ over P is finite.*

Theorem 3.3. *If P is countable, $FORM$ over P is countable.*

Proof. Given $\forall \phi \in FORM, \phi$ is finitely long, this theorem is another way of stating: The language formed from countable alphabet $P \cup C$ with the inductive construction rules, is countable. We can prove a stronger theorem first. □

Theorem 3.4. *All languages formed from a countable alphabet is countable.*

Proof. Consider $L = \Sigma^*$, we will prove that Σ is countable $\rightarrow L$ is countable. We use the same diagnol proof for the countability of $\mathbb{Q}$ to show an enumaration of all strings formable from a countable alphabet. To formalize, we use induction to prove that $\forall i \geq 0, \Sigma^i$ is countable
Base case: $\Sigma^1 = \Sigma$, countable as defined.
Inductive hypothesis: Σ^n is countable: we enumerate all strings in Σ^n, and use the same diagonal proof to show the concatenation of one symbol from the countable alphabet to every string Σ^{n+1} is also countable. Thus we can give an enumeration of all Σ^i and use the diaganol argument again to show the existence of an

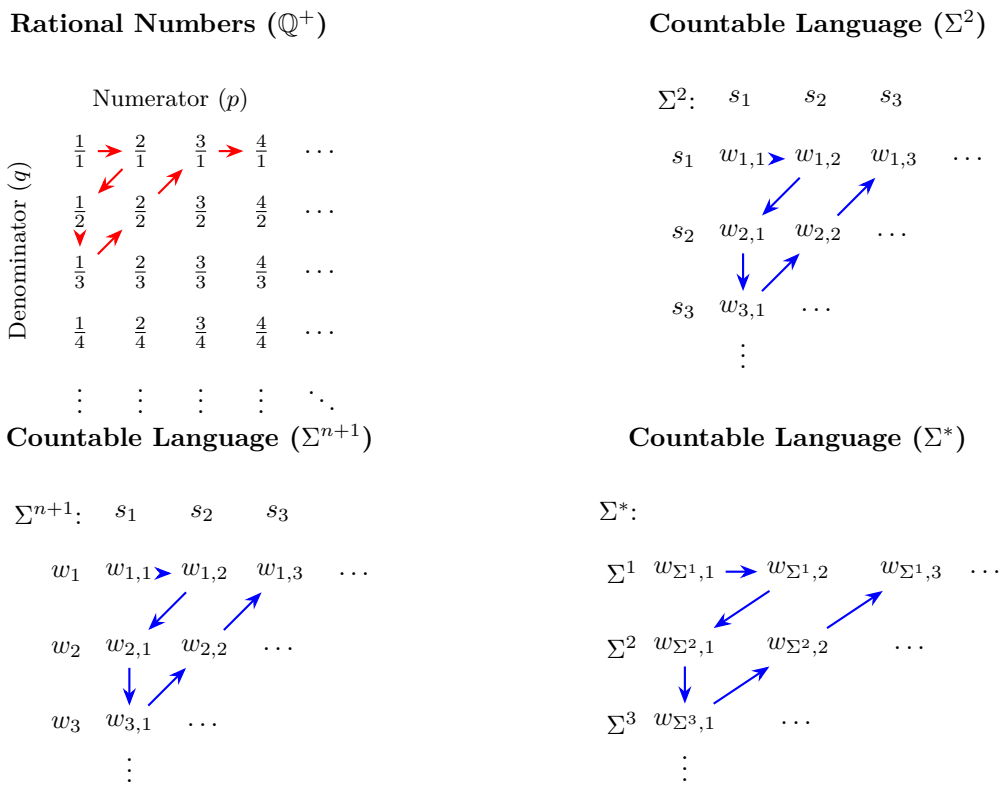


Figure 1: Comparison of diagonal enumeration for numbers vs. strings.

enumeration of Σ^*.
Alternatively, Given

$$\Sigma^* = \bigcup_{i \geq 0} \Sigma^i$$

From the Axiom of Choice, which implies the union of countably many countable set is countable, Σ^* is countable. Since $\forall L$ over $\Sigma, L \subseteq \Sigma^*, \Sigma^*$ is countable, all languages over countable Σ is countable. □

Definition 3.5 (Finitely Satisfiable)**.** *A set of formulas Γ is said to be **finitely satisfiable** if and only if for all finite subsets $\Gamma_0 \subseteq \Gamma$, Γ_0 is satisfiable.*

Theorem 3.6 (Compactness Theorem)**.**

$$\Gamma \textit{ is satisfiable} \iff \Gamma \textit{ is finitely satisfiable}$$

Proof. Consider a special case where Γ is "T-good", defined as:

$$\forall p \in P, (p \in \Gamma \vee \neg p \in \Gamma)$$

Claim: If Γ is T-good, then Γ is finitely satisfiable $\implies$ Γ is satisfiable.
Suppose Γ is T-good and finitely satisfiable. We define a candidate valuation v_L such that for all $p \in P$:

$$v_L(p) = \begin{cases} T & \text{if } p \in \Gamma \\ F & \text{if } \neg p \in \Gamma \end{cases}$$

First, we prove that for every $p \in P$, **exactly one** of $\{p, \neg p\}$ is in Γ. By the T-good property, at least one is in Γ. Suppose both $\{p, \neg p\} \subseteq \Gamma$. Consider the finite subset:

$$\Gamma' = \{p, \neg p\}$$

Since Γ' is a finite subset of a finitely satisfiable set, it must be satisfiable. However, $\neg\exists v$ such that $v \models \{p, \neg p\}$, which is a contradiction. Thus, v_L is well-defined.

Now we show $v_L \models \Gamma$. Suppose $\exists \phi \in \Gamma$ such that $v_L \not\models \phi$. Define the following finite sets based on the atoms occurring in ϕ:

$$\Gamma_\phi^+ = \{p \mid p \in \text{OCCUR}(\phi) \wedge v_L(p) = T\}$$
$$\Gamma_\phi^- = \{\neg p \mid p \in \text{OCCUR}(\phi) \wedge v_L(p) = F\}$$
$$\Gamma_\phi = \{\phi\} \cup \Gamma_\phi^+ \cup \Gamma_\phi^-$$

Since Γ_ϕ is a finite subset of Γ, it must be satisfiable by some valuation v. For this v:

- $\forall p$ such that $p \in \Gamma_\phi^+$, $v(p) = T$.
- $\forall \neg p$ such that $\neg p \in \Gamma_\phi^-$, $v(\neg p) = T \implies v(p) = F$.

By construction, v and v_L agree on all atoms in OCCUR(ϕ). By the **Relevance Lemma**, $v(\phi) = v_L(\phi)$. Since $v \models \Gamma_\phi$, we have $v \models \phi$. This implies $v_L \models \phi$, contradicting our assumption $v_L \not\models \phi$. Therefore, $\forall \phi \in \Gamma, v_L \models \phi$, which implies $v_L \models \Gamma$, Γ is satisfiable.

Now we will generalize this to all Γ which may not be 'T-good', by showing that we can construct a 'T-good' finitely satisfiable set Δ, from any finitely satisfiable Γ, and therefore show that $\Gamma SAT \iff \Delta SAT$.
We will only prove the case for countable P now. Let an enumeration of $p \in P$ be $\{p_1, p_2, \dots\}$
Define finite subset Γ_i of Γ as such:

$$\Gamma_0 = \Gamma$$
$$\Gamma_{i+1} = \begin{cases} \Gamma_i \cup \{p_i\} \text{ if } \Gamma_i \cup \{p_i\} \text{ is finitely satisfiable} \\ \Gamma_i \cup \{\neg(p_i)\} \text{ Otherwise} \end{cases}$$

Here we need to prove that $\forall i$, between $\Gamma_i \cup \{p_i\}$ and $\Gamma_i \cup \{(\neg p_i)\}$ at least one case is also finitely satisfiable.

Theorem 3.7. *Let Γ be finitely satisfiable set, and ϕ be a formula, then $\Gamma \cup \{\phi\}$ is finitely satisfiable or $\Gamma \cup \{(\neg\phi)\}$ is finitely satisfiable.*

Proof. We prove the contrapositive of this statement: if $\Gamma \cup \{\phi\}$ and $\Gamma \cup \{(\neg\phi)\}$ are both not finitely satisfiable, Γ is not fintely satisfiable. Given both are not finitely satisfiable, there exist finite subsets $\Gamma_1, \Gamma_2 \subseteq \Gamma$ such that $\Gamma_1 \cup \{\phi\}$ and $\Gamma_2 \cup \{(\neg\phi)\}$ are not satisfiable. Therefore for all valuations, it must not model either Γ_1 or $\{\phi\}$, and not model either Γ_2 or $\{(\neg\phi)\}$. Denote the set of valuations that models formula ψ as $models(\psi)$, the set of valuations that models set of formula Σ as $models(\Sigma)$

$$\begin{aligned} models(\Gamma_1) \cap models(\phi) = \emptyset \\ &\to \forall v \in models(\Gamma_1), v \not\models \phi \to v \models (\neg\phi) \\ &\to models(\Gamma_1) \subseteq models((\neg\phi)) \\ models(\Gamma_2) \cap models((\neg\phi)) = \emptyset \\ &\to \forall v \in models(\Gamma_2), v \not\models (\neg\phi) \to v \models (\phi) \\ &\to models(\Gamma_2) \subseteq models(\phi) \end{aligned}$$

Now consider the set of valuations that models $\Gamma_1 \cup \Gamma_2$

$$models(\Gamma_1 \cup \Gamma_2) = models(\Gamma_1) \cap models(\Gamma_2)$$
$$(models(\Gamma_1) \cap models(\Gamma_2)) \subseteq (models(\phi) \cap models((\neg\phi))) = \emptyset$$

Since $\Gamma_1 \cup \Gamma_2$ is a finite subset of Γ that is not satisfiable, Γ is not finitely satisfiable. □

Now prove that

Lemma 3.8. *$\forall i, \Gamma_i$ is finitely satisfiable.*

Proof. Base case:

$$\Gamma_0 = \Gamma \text{ is finitely satisfiable by definition}$$

Inducetive hypothesis: Suppose

$$\Gamma_n \text{ is finitely satisfiable}$$

By Lemma 3.7, $\Gamma_n \cup \{p_n\}$ and $\Gamma_n \cup \{(\neg p_n)\}$ there must be at least one that is finitely satisfiable. $\rightarrow \Gamma_{n+1}$ is therefore finitely satisfiable. □

Next we will show that

Lemma 3.9. *$\Gamma_U = \bigcup_{i\geq 0} \Gamma_i$ is finitely satisfiable.*

Proof. Consider finite set $\Gamma' \subseteq \Gamma_U$, since Γ' is finite, and $\forall k \geq 0, \Gamma_k \subseteq \Gamma_{k+1}$ by definition, $\exists j \geq 0, \Gamma' \subseteq \Gamma_j = \bigcup_{i\geq 0}^{j} \Gamma_i$. By Lemma 3.8, Γ_j is satisfiable $\rightarrow \Gamma'$ is finitely satisfiable. □

Lemma 3.10. *Γ_U is satisfiable.*

Proof. From the construction, $\forall i \geq 0$ at least one of p_i and $(\neg p_i)$ is in $\bigcup_{i\geq 0} \Gamma_i$. Since $\bigcup_{i\geq 0} \Gamma_i$ is finitely satisfiable, $\forall i \geq 0, p_i$ and $(\neg p_i)$ cannot both be in $\bigcup_{i\geq 0} \Gamma_i$. Otherwise $\{p_i\} \cup \{(\neg p_1)\} \subseteq \bigcup_{i\geq 0} \Gamma_i$ is not satisfiable, contradiction. □

Now we have proven that the construction proposed from a finitely satisfiable formula set Γ to $\bigcup_{i\geq 0} \Gamma_i$ is also finitely satisfiable. Now to complete the proof of the compactness theorem, we need to prove that $\exists v, v \models \Gamma$.

Proof. From Lemma 3.10, $\forall i \geq 0, (p_i \in \Gamma_U) \oplus ((\neg p_i) \in \Gamma_U)$, $\exists v, v \models \Gamma_U$

$$v(p) : P \mapsto \{T, F\} = \begin{cases} T, & \text{if } p \in \Gamma_U \\ F, & \text{if } (\neg p) \in \Gamma_U \end{cases}$$

Define the literal set $Literals = \{p_i, (\neg p_i) \mid i \geq 0\}$, and function

$$\tau(p) : P \mapsto Literals = \begin{cases} p, & \text{if } p \in \Gamma_U \\ (\neg p), & \text{if } (\neg p) \in \Gamma_U \end{cases}$$

Lemma 3.11.

$$v \models \Gamma_U \rightarrow v \models \tau(\Gamma)$$

Let $\phi \in \Gamma$, Define literal set $X = \{\tau(p) \mid p \in OCCUR(\phi)\}$ where X is a set containing single proposition literals. By definition we have

$$X \subseteq \Gamma_U \rightarrow v \models X \rightarrow \forall \psi \in X, v(\psi) = T$$

and $OCCUR(X) = OCCUR(\phi)$ Since both X is finite, $X \cup \{\phi\}$ is a finite subset to Γ_U, thus satisfiable. $\rightarrow \exists v', v' \models X \cup \{\phi\} \rightarrow v' \models X, v' \models \phi \rightarrow \forall \psi \in X, v'(\psi) = T$.

$$\forall p \in OCCUR(\phi), v(p) = v'(p), v' \models \phi \rightarrow v \models \phi$$

Therefore, the valuation that models enlarged construction Γ_U also models Γ, thus Γ is satisfiable. □

□

3.1 Application of the Compactness theorem for proposition logics

We have already shown that we can use propositional logic to model computational problems, the compactness theorem allows us to extend problem modelling to instances of infinite size. Recall the k-colorable problem, where we used propositional logic to model any finite size instance.

Theorem 3.12. *A graph is k-colorable iff every finite subgraph is k-colorable.*

Definition 3.13 (Infinite graph). *$G = (V, E)$ is an infinite graph if V is infinite.*

Definition 3.14 (Subgraph). *$G' = (V', E')$ is a subgraph to G if $V' \subseteq V, E' \subseteq E$*

Definition 3.15 (Induced subgraph). *$G' = (V', E')$ is an induced subgraph to G if G' is a subgraph to G, and $\{u, v\} \in E' \iff \{u, v\} \in E \wedge u, v \in V'$*

Proof. Since we have formulated any finite instance of k-colorable graph as propositional logic formula, we can define infinite set

$$\Gamma_G = \{\phi_{G'}^{k-colorable} \mid G' \text{ is finite subgraph of } G\}$$

We will first prove that

Lemma 3.16. *G is finitely k-colorable $\rightarrow \Gamma_G$ is finitely satisfiable.*

Proof. We will reuse the proposition definition from last chapter, where $|V|$ and thus P is countable

$$P = \{p_{v,i} \mid v \in V, i \in [1, k]\}$$

For each instance on $G = (V, E)$ and k colors, we formulate as such to enforce constraints on edges (No same color on adjacent vertices), and vertices (exactly 1 coloring on each vertex).

$$\phi = \phi^{nbr} \wedge \phi^{exactly-1-color}$$

Consider the infinite graph $G = (V, E)$

$$\begin{aligned}
\Gamma_G &= \Gamma_G^{nbr} \cup \Gamma_G^{exactly-1-color} \\
\Gamma_G^{exactly-1-color} &= \{\phi_v^{exactly-1-color} \mid v \in V\} \\
\phi_v^{exactly-1-color} &= (\bigvee_{i \in [1,k]} p_{v,i}) \wedge (\bigwedge_{i \neq j \in [1,k]} (p_{v,i} \rightarrow p_{v,j})) \\
\Gamma_G^{nbr} &= \{\phi_{\{u,v\}}^{nbr} \mid \{u, v\} \in E\} \\
\phi_{\{u,v\}}^{nbr} &= \bigwedge_{i \in [1,k]} (p_{u,i} \rightarrow p_{v,i})
\end{aligned}$$

Suppose the contrary, where $G = (V, E)$ is finitel k-colorable, Γ_G is not finitely satisfiable, $\exists$ finite subset $\Gamma_0 \subseteq \Gamma_G, \Gamma_0$ is not satisfiable. Let $G_0 = (V_0, E_0)$ be a finite subgraph of G, such that

$$V_0 = \{v \mid v \in V, \phi_v^{exactly-1-color} \in \Gamma_0\} \cup \{u \mid u \in V, \exists u', \phi_{\{u,u'\}}^{nbr} \in \Gamma_0\}$$

$$E_0 = \{\{u, v\} \mid \{u, v\} \in E, u, v \in V_0\}$$

Here we are formulating an instance of finite k-colorable induced subgraph of G from finite subset Γ_0. Now we need to show that G_0 is not k-colorable to reach the contradiction, with another contradiction.
Suppose G_0 is k-colorable, let C_0 be a k-coloring of G_0, define valuation

$$v_0(p_{u,i}) = \begin{cases} T, \text{ if } C_0(u) = i \\ F, \text{ if } C_0(u) \neq i \end{cases}$$

Let

$$\Gamma_0' = \{\phi_v^{exactly-1-color} \mid v \in V_0\} \cup \{\phi_{\{u,v\}}^{nbr} \mid \{u, v\} \in E_0\}$$

Consider

$$\Gamma_0 \subseteq \Gamma_0'$$

Since Γ_0' is the same construction from k-colorability to formuala, G_0 is k-colorable, we have

$$v_0 \models \Gamma_0'$$

By relevance lemma

$$v_0 \models \Gamma_0$$

Contradicting to our initial assumption that Γ_0 is NOT satisfiable. □

By Compactness theorem, Γ_G is finitely satisfiable $\rightarrow \Gamma_G$ is satisfiable.

Lemma 3.17. *Γ_G is satisfiable $\rightarrow$ G is k-colorable*

Proof. Γ_G is satisfiable $\rightarrow \exists v, v \models \Gamma_G$. We can construct a coloring C from v similarly to how we constructed v_0 from C_0 from previous proof:

$$C(v) = i, \text{ if } v(p_{v,i}) = T$$

By construction the coloring is valid, detailed technicality is ommitted. □

□

4 Proof System

4.1 Hilbert Style Proof System

Four axiom schema of the hilberts style proof system. Axiom schema means that the ϕ, ψ, ρ in these schema are placehoders that can be replaced by any valid formula

1. Axiom 1: Simplification

$$\frac{}{p \rightarrow (q \rightarrow p)}$$

2. Axiom 2: Distribution

$$\frac{}{(p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))}$$

3. Axiom 3: Double Negation Elimination (DNE)

$$\frac{}{((p \rightarrow \bot) \rightarrow \bot) \rightarrow p}$$

4. Rule of Inference: Modus Ponens

$$\frac{p \rightarrow q \qquad p}{q}$$

The horizontal lines are judgement lines, which means that whats below can be inferred from whats above. The first 3 rules has no assumptions or premises, meaning they are ways we can introduce arbituary claims or assumption into our proofs.

Definition 4.1 (Proof)**.** *A proof of the consequence of ϕ from Γ is a finite sequence of formula $c_1, c_2, \ldots, c_k$ where $\forall i \leq k, c_i \in \Gamma$ or, c_i is obtained using one of the 4 axioms:*

- $\exists j, l < i, c_j \equiv c_l \rightarrow c_i$
- $\exists \phi, \psi, c_i \equiv \phi \rightarrow (\psi \rightarrow \phi)$
- $\exists \phi, \psi, \rho, c_i \equiv [\phi \rightarrow (\psi \rightarrow \rho)] \rightarrow [(\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \rho)]$
- $\exists \phi, c_i \equiv ((\phi \rightarrow \bot) \rightarrow \bot) \rightarrow \phi$

Definition 4.2 (Derivable)**.** *ϕ is derivable from Γ if there is a proof in HSPS of "ϕ is a consequence of Γ"*

$$\Gamma \vdash \phi$$

Example 4.3.

$$\Gamma = \{p, p \rightarrow q\}, \quad \phi = q$$

Proof. We can construct proof sequence:

$c_1 = p$	$(p \in \Gamma)$
$c_2 = p \to q$	$(p \to q \in \Gamma)$
$c_3 = q$	(Modus Ponens on c_1, c_2)

□

Sometimes HSPS requires us to creatively introduce new hypothesis to assist in the proof.

Example 4.4.

$$\Gamma = \{\bot\}, \phi = p$$

Intuitively this does not sound provable, just from $\bot$, but axioms that require no premises allows us to introduce helper hypothesis. In this case $(p \to \bot)$

Proof.

$c_1 = \bot$	$(\bot \in \Gamma)$
$c_2 = \bot \to [(p \to \bot) \to \bot]$	(Axiom 1: Simplification)
$c_3 = (p \to \bot) \to \bot$	(Modus Ponens on $\bot, (p \to \bot)$)
$c_4 = [(p \to \bot) \to \bot] \to p$	(Axiom 3: Double negation)
$c_5 = p$	(Modus Ponens on $c_3 \equiv [(p \to \bot) \to \bot], c_4 \equiv ([(p \to \bot) \to \bot] \to p)$)

□

As seen from this example, HSPS can get convoluted and unintuitive for relatively straightforward consequences.

1. Self-Implication: $\vdash p \to p$

Idea. We must simulate identity using only distribution.

1. $p \to ((p \to p) \to p)$ (Axiom 1)
2. $p \to (p \to p)$ (Axiom 1)
3. $(p \to ((p \to p) \to p)) \to ((p \to (p \to p)) \to (p \to p))$ (Axiom 2)
4. $(p \to (p \to p)) \to (p \to p)$ (MP 1,3)
5. $p \to p$ (MP 2,4)

Even the trivial tautology requires a nontrivial derivation.

2. Contraposition: $\vdash (p \to q) \to ((q \to \bot) \to (p \to \bot))$

Idea. Assume $p \to q$ and $q \to \bot$; derive $p \to \bot$ via Axiom 2.

1. $(p \to (q \to \bot)) \to ((p \to q) \to (p \to \bot))$ (Axiom 2)
2. $(q \to \bot) \to (p \to (q \to \bot))$ (Axiom 1)
3. $(p \to q) \to ((q \to \bot) \to (p \to \bot))$

The last line follows by two applications of Modus Ponens using (1) and (2).
This shows implication reversal must be encoded indirectly.

3. Peirce's Law: $\vdash ((p \to q) \to p) \to p$

This formula characterizes classical logic.

Sketch.

1. $((p \to q) \to p) \to (((p \to q) \to p) \to ((p \to q) \to p))$ (Ax1)
2. Use Axiom 2 repeatedly to simulate assumption chaining.
3. Derive $(((p \to q) \to p) \to \bot) \to \bot$.
4. Apply Axiom 3 (DNE) to conclude $((p \to q) \to p) \to p$.

The essential trick: Peirce's law is obtained by embedding the formula into a double-negation context and applying Axiom 3.

This is far from syntactically obvious.

4. Explosion: $\vdash \bot \to p$

In this system $\bot$ is primitive via $p \to \bot$.

Idea. Show that from $\bot$ we can derive anything.

1. $p \to (\bot \to p)$ (Ax1)
2. $\bot \to (p \to \bot)$ (Ax1)
3. $(\bot \to (p \to \bot)) \to ((\bot \to p) \to (\bot \to \bot))$ (Ax2)
4. Derive $\bot \to \bot$ using the $p \to p$ pattern.
5. Conclude $\bot \to p$.

Notice how even explosion must be simulated through distribution.

5. Double Negation Introduction: $\vdash p \to ((p \to \bot) \to \bot)$

Although Axiom 3 gives elimination, introduction must be derived.

1. $(p \to \bot) \to (p \to (p \to \bot))$ (Ax1)
2. Use Axiom 2 to rearrange implication nesting.
3. Conclude $p \to ((p \to \bot) \to \bot)$.

This illustrates asymmetry between DNE and its converse.

4.2 Metatheorems

4.2.1 Monotonicity theorem

Theorem 4.5.

$$\Gamma \vdash A, \Gamma \subseteq \Delta \implies \Delta \vdash A$$

Proof. Let the proof of A from Γ be the sequence $\langle \phi_1, \phi_2, \ldots, \phi_n \rangle, \phi_n = A$

Base case: $n = 1$

- Case 1: $\phi_1 \in \Gamma \implies \phi_1 \in \Delta \implies \Delta \vdash \phi_1$
- Case 2: ϕ_1 is from one of the axiom schema, $\Delta \vdash \phi_1$

Inductive step: $\forall i < n, \Gamma \vdash \phi_i, \Delta \vdash \phi_i$

- Case 1, 2: $\phi_{i+1} \in \Gamma$ or ϕ_{i+1} is from one of the axiom schema, trivial
- Case 3: ϕ_{i+1} is from Modus Ponens, $\exists \phi_j$ such that

$$\frac{\phi_j \to \phi_{i+1} \qquad \phi_j}{\phi_{i+1}}$$

$\Gamma \vdash \phi_j, \Gamma \vdash \phi_j \to \phi_{i+1}$ by inductive hypothesis, $\Delta \vdash \phi_j, \Delta \vdash \phi_j \to \phi_{i+1}$, by MP, $\Delta \vdash \phi_{i+1}$

□

4.2.2 Deduction

Theorem 4.6.

$$\Gamma \cup \{A\} \vdash B \iff \Gamma \vdash A \to B$$

Proof. ($\implies$)

We will use structural induction over the length of derivation from $\Gamma \cup \{A\}$ to B, $\Gamma \cup \{A\} \vdash B \implies \exists \langle \phi_1, \phi_2, \ldots, \phi_n \rangle, \phi_n = B$.

Base case: $n = 1, \phi_n = B$

- Case 1: B is an axiom $\Gamma \vdash B$. By simplification axiom, $\Gamma \vdash B \to (A \to B)$, by MP, $\Gamma \vdash A \to B$.
- Case 2: $B = A$, we will use the simplification and distribution axioms:
  - $p \equiv A$
  - $q \equiv (A \to A)$
  - $r \equiv A$
  - $(A \to ((A \to A) \to A)) \to ((A \to (A \to A)) \to (A \to A))$ (Distribution)
  - $A \to ((A \to A) \to A)$ (Simplification)
  - $(A \to (A \to A)) \to (A \to A)$ (MP)
  - $A \to (A \to A)$ (Simplification)
  - $(A \to A)$ (MP)

  $\Gamma \vdash A \to A \implies \Gamma \vdash A \to B$

Inductive step: Suppose $\forall i < n, \Gamma \cup \{A\} \vdash \phi_i \implies \Gamma \vdash A \to \phi_i$

- Case 1, 2, 3: ϕ_{i+1} is in Γ or ϕ_{i+1} is from one of the axioms, trivial; If $\phi_{i+1} = A$, the same as Case 2 in base case
- Case 4: ϕ_{i+1} is obtained from MP, where $\exists j \leq i$,

$$\begin{aligned} &\Gamma \cup \{A\} \vdash \phi_j && \implies \Gamma \vdash A \to \phi_j \\ &\Gamma \cup \{A\} \vdash \phi_j \to \phi_{i+1} && \implies \Gamma \vdash A \to (\phi_j \to \phi_{i+1}) \end{aligned}$$

$$\frac{\phi_j \to \phi_{i+1} \qquad \phi_j}{\phi_{i+1}}$$

$$\begin{aligned} &\Gamma \vdash (A \to \phi_j) \to (A \to \phi_{i+1}) && \text{by distribution} \\ &\Gamma \vdash A \to \phi_{i+1} && \text{by MP} \end{aligned}$$

□

Theorem 4.7 (Soundness of HSPS). *HSPS is sound.*

$$\Gamma \vdash \phi \implies \Gamma \models \phi$$

Proof. Define property $H(i) : \forall j \leq i, \Gamma \models c_j$.

Base case: $i = 1$

Suppose $\Gamma \vdash \phi, \exists \pi = c_1, c_2, \ldots, c_k, c_k = \phi$. By definition, $c_1 \in \Gamma$ or

- $\exists \phi, \psi, c_1 \equiv \phi \to (\psi \to \phi)$
- $\exists \phi, \psi, \rho, c_1 \equiv [\phi \to (\psi \to \rho)] \to [(\phi \to \psi) \to (\phi \to \rho)]$
- $\exists \phi, c_1 \equiv ((\phi \to \bot) \to \bot) \to \phi$

Note that Modus Ponens is not possible to yield c_1 as we need at least 1 previous formula.
If $c_1 \in \Gamma, \Gamma \models c_1$; Else, if c_1 comes from any other axiom, c_1 is valid, i.e. $\forall v, v \models c_1$, therefore $\Gamma \models c_1$

Inductive step:

Suppose $H(i), \forall j < i, \Gamma \models c_j, \forall v \models \Gamma, v \models c_j$.

- Case 1, 2: $c_{i+1} \in \Gamma$, or c_{i+1} is from an axiom, $\Gamma \models c_{i+1}, H(i+1)$
- Case 3: c_{i+1} comes from MP: $\exists j < i, \Gamma \vdash c_j, \Gamma \vdash c_j \to c_{i+1}$. By inductive hypothesis, $\forall v \models \Gamma, v \models c_j, v \models c_j \to c_{i+1}$

$$\begin{aligned} &v(c_j) = T \\ &v(c_j \to c_{i+1}) = T \equiv v(\neg c_j \vee c_{i+1}) = T \\ &\quad v(\neg c_j) = T \text{ OR } v(c_{i+1}) = T \\ &v(c_j) = T \implies v(\neg c_j) = F \implies v(c_{i+1}) = T \\ &\implies \forall v \models \Gamma, v \models c_{i+1} \implies \Gamma \models c_{i+1} \end{aligned}$$

□

Theorem 4.8 (Completeness of HSPS). *HSPS is complete.*

$$\Gamma \models \phi \implies \Gamma \vdash \phi$$

4.3 Weak Completeness and Completeness of HSPS

We work with the Hilbert-Style Propositional System (HSPS):

- Axiom 1: $p \to (q \to p)$
- Axiom 2: $(p \to (q \to r)) \to ((p \to q) \to (p \to r))$
- Axiom 3: $((p \to \bot) \to \bot) \to p$
- Rule: Modus Ponens

We write $\vdash \varphi$ for provability and $\models \varphi$ for semantic validity.

1. Weak Completeness

Theorem 4.9 (Weak Completeness). *If* $\models \varphi$, *then* $\vdash \varphi$.

Idea. We prove the contrapositive: if $\nvdash \varphi$, then $\not\models \varphi$. That is, if φ is not provable, then there exists a valuation falsifying it.

Step 1: Consistency and Maximal Consistency. A set Γ is *consistent* if $\Gamma \nvdash \bot$.
If $\nvdash \varphi$, then $\{\neg\varphi\}$ is consistent.
Using a standard Lindenbaum construction:

- Extend any consistent set Γ
- to a *maximal consistent set* Δ
- such that for every formula ψ:

$$\psi \in \Delta \quad \text{or} \quad \neg\psi \in \Delta.$$

This construction relies only on classical logic and the deduction theorem (derivable in HSPS).

Step 2: Canonical Valuation. Define a valuation v_Δ by:

$$v_\Delta(p) = \begin{cases} \text{true} & \text{if } p \in \Delta \\ \text{false} & \text{otherwise} \end{cases}$$

We prove by structural induction:

$$\psi \in \Delta \iff v_\Delta \models \psi.$$

The key steps use:

- Maximality of Δ
- Closure under Modus Ponens
- Axiom 3 for classical negation

Step 3: Falsifying φ. Since $\neg\varphi \in \Delta$, we obtain:

$$v_\Delta \models \neg\varphi, \quad \text{hence} \quad v_\Delta \not\models \varphi.$$

Therefore φ is not valid.

Conclusion.

$$\models \varphi \implies \vdash \varphi.$$

□

2. Strong Completeness (Full Completeness)

Theorem 4.10 (Completeness)**.** *If* $\Gamma \models \varphi$*, then* $\Gamma \vdash \varphi$*.*

Reduction to Weak Completeness. Suppose $\Gamma \models \varphi$.
Then:

$$\models \bigwedge \Gamma \to \varphi.$$

If Γ is finite, we directly apply weak completeness:

$$\vdash (\bigwedge \Gamma) \to \varphi.$$

Repeated applications of Modus Ponens yield:

$$\Gamma \vdash \varphi.$$

Infinite Case. If Γ is infinite:

- If $\Gamma \models \varphi$,
- then some finite subset $\Gamma_0 \subseteq \Gamma$ satisfies $\Gamma_0 \models \varphi$ (compactness of propositional logic).

Apply the finite case to Γ_0, then use monotonicity of derivability:

$$\Gamma \vdash \varphi.$$

3. Conceptual Structure

Weak completeness:

$$\models \varphi \Rightarrow \vdash \varphi.$$

Strong completeness:

$$\Gamma \models \varphi \Rightarrow \Gamma \vdash \varphi.$$

The essential engine is:

1. Maximal consistent sets
2. Canonical valuation
3. Truth lemma

Axiom 3 ensures classicality, making the canonical model work.

4. Final Picture

Soundness:

$$\vdash \varphi \Rightarrow \models \varphi.$$

Completeness:

$$\models \varphi \Rightarrow \vdash \varphi.$$

Hence:

$$\vdash \varphi \iff \models \varphi.$$

HSPS is therefore sound and complete for classical propositional logic.

■

4.4 Resolution Proof System

Definition 4.11 (Clause)**.** *For each proposition p define literals p and $\neg p$. A clause C is finite set of literals over the proposition set P.*

$$v \models C = \{\ell_1, \ell_2, \ldots, \ell_k\} \iff \exists j, v \models \ell_j$$

As such, a cluase C is satisfiable if $\exists v, v \models C$

Note that from this defintion, an empty clause (an empty set of literals) is not satisfiable, as intuitively, a valuation models a clause if there exists at least one literal in the clause such that $v \models \ell$.

Definition 4.12 (Conjunctive normal form (CNF))**.** *A formula ϕ is said to be in Conjunctive Normal Form is*

$$\phi \equiv \bigwedge_{i=1}^{k} C_i$$

Definition 4.13 (Satisfiability of a set of clauses)**.**

$$\Gamma = \{C_1, C_2, \dots\}$$

- Γ *is satisfiable if* $\exists v, \forall C \in \Gamma, v \models C$
- Γ *is unsatisifiable if* $\forall v, \exists C \in \Gamma, v \not\models C$

By this definition,

- $\Gamma = \emptyset$ is vacuously satisfiable
- $\Gamma = \{\emptyset\}$ is not satisifiale.
- Any set of clauses that contains the empty set is unsatisfiable

Theorem 4.14. *Each well-formed formula can be transformed to a logically equivalent CNF*

Proof. **Base case:** $\phi \in P$

trivial

Inductive Case 1: $\neg$

Suppose there exists a CNF $\bigwedge_i \bigvee_j \ell_{i,j}$ logically equivalent to ϕ.

$$\begin{aligned}
\neg\phi &= \neg((\ell_{1,1} \vee \ell_{1,2} \vee \dots) \wedge (\ell_{2,1} \vee \ell_{2,2} \vee \dots) \wedge \dots) \\
&= \neg(\ell_{1,1} \vee \ell_{1,2} \vee \dots) \vee \neg(\ell_{2,1} \vee \ell_{2,2} \vee \dots) \vee \dots \\
&= (\neg\ell_{1,1} \wedge \neg\ell_{1,2} \wedge \dots) \vee (\neg\ell_{2,1} \wedge \neg\ell_{2,2} \wedge \dots) \vee \dots \\
&= (\neg\ell_{1,1} \vee \neg\ell_{2,1} \vee \dots) \wedge ()
\end{aligned}$$

Inductive Case 2: $\wedge$

Suppose $\phi \equiv \bigwedge_{i \in I} C_i$ and $\psi \equiv \bigwedge_{j \in J} D_j$ are CNFs.

$$\begin{aligned}
\phi \wedge \psi &\equiv \left(\bigwedge_{i \in I} C_i\right) \wedge \left(\bigwedge_{j \in J} D_j\right) \\
&\equiv \bigwedge_{k \in I \cup J} E_k
\end{aligned}$$

where E_k is a clause from the set of clauses of ϕ or ψ. Since the conjunction of two conjunctions is simply a larger conjunction of the original clauses, the result is by definition a CNF.

Inductive Case 3: $\vee$

Suppose $\phi \equiv \bigwedge_{i \in I} C_i$ and $\psi \equiv \bigwedge_{j \in J} D_j$ are CNFs. By applying the Distributive Law:

$$\begin{aligned}
\phi \vee \psi &\equiv \left(\bigwedge_{i \in I} C_i\right) \vee \left(\bigwedge_{j \in J} D_j\right) \\
&\equiv \bigwedge_{i \in I} \left(C_i \vee \left(\bigwedge_{j \in J} D_j\right)\right) \\
&\equiv \bigwedge_{i \in I} \left(\bigwedge_{j \in J} (C_i \vee D_j)\right) \\
&\equiv \bigwedge_{i \in I, j \in J} (C_i \vee D_j)
\end{aligned}$$

Since each C_i and D_j are disjunctions of literals, their union $C_i \vee D_j$ is also a disjunction of literals, and thus a valid clause. The resulting expression is a conjunction of these clauses, satisfying the definition of a CNF.

Conclusion

By the principle of induction on the complexity of the formula, we have shown that if the sub-formulas of ϕ have equivalent CNFs, then $\neg\phi$, $\phi \wedge \psi$, and $\phi \vee \psi$ also have equivalent CNFs. Since the set of connectives $\{\neg, \wedge, \vee\}$ is functionally complete, every well-formed formula can be transformed into a logically equivalent CNF. We can bring all negations to literal level, and supply an additional negation to each literal to negate the whole formula. □

The Resolution Rule

The system relies on a single rule of inference. Given two clauses C' and D' such that $C' = C \oplus \{p\}$ and $D' = D \oplus \{\neg p\}$ (where $\oplus$ denotes a disjoint union), the **resolvent** with respect to p is defined as the clause:

$$Res(C', D') = C \cup D$$

Resolution Proofs

A resolution proof of Γ is a finite sequence of clauses $\pi = C_1, C_2, \ldots, C_n$ where the final clause C_n is the empty clause $\{\}$. For every clause C_i in the sequence, one of the following must be true:

1. C_i is an original clause from the set Γ ($C_i \in \Gamma$).
2. C_i is the resolvent of two previous clauses C_j and C_k (where $j, k < i$) with respect to some proposition p.

Soundness and Completeness

Theorem 4.15 (Soundness)**.** *The resolution proof system is sound:*

$$\Gamma \vdash_{RES} \square \implies \Gamma \textit{ is unsatisfiable}$$

Proof. Suppose there exists a resolution derivation $C_1, C_2, \ldots, C_k$ where $C_k = \square$. We proceed by induction on the length of the derivation k.

Base Case: $k = 1$

If the empty clause $\square$ is in the derivation at step 1, then $\square \in \Gamma$. Since an empty clause is by definition unsatisfiable (it is an empty disjunction), Γ is unsatisfiable.

Inductive Step

Assume any clause derived in $i \leq k$ steps is a semantic consequence of Γ. Consider C_{k+1} derived from C_a and C_b ($a, b \leq k$) using the resolution rule on literal p:

$$C_a = \{p\} \cup C'_a, \quad C_b = \{\neg p\} \cup C'_b, \quad C_{k+1} = C'_a \cup C'_b$$

By the Inductive Hypothesis, $\Gamma \models C_a$ and $\Gamma \models C_b$. Let v be a valuation satisfying Γ.

- If $v(p) = F$, then since $v(C_a) = T$, it must be that $v(C'_a) = T$.
- If $v(p) = T$, then since $v(C_b) = T$, it must be that $v(C'_b) = T$.

In either case, $v(C'_a \cup C'_b) = T$. Thus $\Gamma \models C_{k+1}$. If $C_{k+1} = \square$, then $\Gamma \models \bot$, meaning Γ has no model and is unsatisfiable. □

Theorem 4.16 (Completeness)**.** *The resolution proof system is complete:*

$$\Gamma \textit{ is unsatisfiable} \implies \Gamma \vdash_{RES} \square$$

Proof. We prove this by induction on the number of distinct propositional variables n in Γ.

Base Case: $n = 0$

If Γ is unsatisfiable and has 0 variables, it must contain the empty clause $\square$. Thus $\Gamma \vdash_{RES} \square$.

Inductive Step

Pick a variable p and define two sets:

- $\Gamma_{p=T}$: The set of clauses obtained by setting p to True (remove clauses containing p, remove $\neg p$ from remaining).
- $\Gamma_{p=F}$: The set of clauses obtained by setting p to False (remove clauses containing $\neg p$, remove p from remaining).

If Γ is unsatisfiable, then both $\Gamma_{p=T}$ and $\Gamma_{p=F}$ are unsatisfiable. By the Inductive Hypothesis, there exist resolution proofs for $\square$ from both sets. By re-inserting p and $\neg p$, these proofs yield $\Gamma \vdash_{RES} \{p\}$ and $\Gamma \vdash_{RES} \{\neg p\}$. Resolving these two results yields $\square$. $\square$

5 Complexity

5.1 Turing Machine

Definition 5.1 (Turing Machine)**.**

$$\mathcal{M} = \langle Q, \Sigma, \Gamma, \delta, q_0, q_{Acc}, q_{Rej}, k \rangle$$

- Q*: the set of states*
- Σ*: finite input alphabet*
- Γ*: finite worktap alphabet,* $\square \in \Gamma$
- δ*: the set of transitions*
- q_0*: starting state,* $q_0 \in Q$
- q_{Acc}*: accepting state,* $q_{Acc} \in Q$
- q_{Rej}*: rejecting state,* $q_{Rej} \in Q$ *(Optional)*
- k*: the number of worktaps (Optional)*

A TM has 1 read-only input tape, may have 1 write only output tape, and k work tape. We can show that any finitely many k tapes is equivalent to 1 worktap TM. [TODO]

Transifiton function

$$\delta = \langle (Q \times \Sigma \times \Gamma^k) \times (Q \times \{L, R, S\} \times \Gamma^k \times \{L, R, S\}^k \times \Gamma \times \{L, R, S\}) \rangle$$

A state transition

$$\tau = \langle q, \sigma, \gamma_1, \gamma_2, \ldots, \gamma_k, q', d, \gamma'_1, \gamma'_2, \ldots, \gamma'_k, d_1, d_2, \ldots, d_k, \gamma', d_o \rangle$$

$$\begin{aligned} \tau = \langle & q, \sigma, \\ & \gamma_1, \gamma_2, \ldots, \gamma_k, \\ & q', d, \\ & \gamma'_1, \gamma'_2, \ldots, \gamma'_k, \\ & d_1, d_2, \ldots, d_k, \\ & \gamma', d_o \rangle \end{aligned}$$

Represents a transition from state q*, when the head reading input symbol* σ*, and worktap symbol* γ_i *on worktape* i*, transition into state* q'*, write worktape symbol* γ'_i*, move the input head in the direction* d*, and workhead* i *in direction* d_i*, writing* γ' *on the output tape and move output head in direction* d_o

Non-determiniestic TM

Non deterministic TM has non-deterministic transition funcitons

$$\delta \subseteq \langle (Q \times \Sigma \times \Gamma^k) \times (Q \times \{L, R, S\} \times \Gamma^k \times \{L, R, S\}^k \times \Gamma \times \{L, R, S\}) \rangle$$

5.2 Semantics of TM

Definition 5.2 (Configuration)**.** *A configuration* C_t *of a Turing machine at step* t *captures the overall state of the TM. Now we only consider TM with no rejecting states and output tape.*

$$\begin{aligned} C = \langle & q, \\ & w^1_{in} \triangledown w^2_{in}, \\ & w^1_{w1} \triangledown w^2_{w1} \\ & w^1_{w2} \triangledown w^2_{w2} \\ & \ldots \\ & w^1_{wk} \triangledown w^2_{wk} \rangle \end{aligned}$$

Where $w^1 \triangledown w^2 = w$*, the symbol read by the head at each configuration is the first symbol after* $\triangledown$*. Note*

that ∇ is not a Symbol but just an indication of the head position.
The initial configuration will starts at the starting state q_0, input head and work heads all at the beginning, and all worktapes empty.

$$C_0 = \langle q_0, \nabla w_{in}, \underbrace{\nabla, \nabla, \ldots, \nabla}_{k}, \rangle$$

Configuration

$$\begin{aligned} C_{t+1} = \langle q, & \\ & w_{in}^1 \nabla w_{in}^2, \\ & w_{w1}^1 \nabla w_{w1}^2 \\ & w_{w2}^1 \nabla w_{w2}^2 \\ & \ldots \\ & w_{wk}^1 \nabla w_{wk}^2 \rangle \end{aligned}$$

is a valid successor of

$$\begin{aligned} C_t = \langle q', & \\ & w_{in}^{1'} \nabla w_{in}^{2'}, \\ & w_{w1}^{1'} \nabla w_{w1}^{2'} \\ & w_{w2}^{1'} \nabla w_{w2}^{2'} \\ & \ldots \\ & w_{wk}^{1'} \nabla w_{wk}^{2'} \rangle \end{aligned}$$

$$\exists \tau = \langle p, \sigma, \gamma_1, \gamma_2, \ldots, \gamma_k, p', d, \gamma_1', \gamma_2', \ldots, \gamma_k', d_1, d_2, \ldots, d_k, \gamma', d_o \rangle$$

such that:

- *σ is the first symbol after the input head: $w_{in}^2[0]$*
- *γ_i is the first symbol after work head i : $w_{wi}^2[0]$*
- – *if $d = S : w_{in}' = w_{in}$*
  $|w_{in}^1| - |w_{in}'^1| = 0$
  – *if $d = L : w_{in}' = w_{in}^1[:-1] \nabla w_{in}^1[-1] \cdot w_{in}^2$*
  $|w_{in}^1| - |w_{in}'^1| = 1$
  – *if $d = L : w_{in}' = w_{in}^1 \cdot w_{in}^2[0] \nabla w_{in}^2[1:]$*
  $|w_{in}^1| - |w_{in}'^1| = -1$
- *$\forall i \in [1, k]$,*
  – *if $d_i = S : w_{wi}' = w_{wi}$*
  $|w_{wi}^1| - |w_{wi}'^1| = 0$
  – *if $d_i = L : w_{wi}' = w_{wi}^1[:-1] \nabla w_{wi}^1[-1] \cdot w_{wi}^2$*
  $|w_{wi}^1| - |w_{wi}'^1| = 1$
  – *if $d_i = L : w_{wi}' = w_{wi}^1 \cdot w_{wi}^2[0] \nabla w_{wi}^2[1:]$*
  $|w_{wi}^1| - |w_{wi}'^1| = -1$

Definition 5.3. *$\forall w \in \Sigma^*$, $\mathcal{M}$ yields some valid sequences of configuratures, which is called a run $\pi_w = C_0, C_1, \ldots$ where*

- *C_0 is the initial configuration*
- *$\forall t \geq 1, C_t$ is a valid successsor of C_{t-1}*

A run is accepting if

1. *π is finite, and*

2. π *ends with a configurration* C_n, *where* $C_n.q = q_{Acc}$

A run is rejecting if

- π *is infinite, or*
- π *ends with a configurration* C_n, *where* $C_n.q \neq q_{Acc}$

Definition 5.4. $w \in \Sigma*$ *is accepted by* $\mathcal{M}$ *if* $\exists \pi_w, \pi_w$ *is accepting.*

Now we can bridge to the concept of languages,

Definition 5.5. *The set of strings accepted by* $\mathcal{M}$ *is the language described by* $\mathcal{M}$, *ie*

$$L(\mathcal{M}) = \{w \in \Sigma^* \mid w \text{ is accepted by } \mathcal{M}\}$$

Now we will focus on decidable languages, and their recognizoing non deterministic TM.

Definition 5.6 (Terminating Turing Machine)**.** $\mathcal{M}$ *is termeinating if* $\forall w \in \Sigma^*, \forall \pi_w, \pi_w$ *is finite.*

5.3 Running time and Space

Definition 5.7.

$$T : \mathbb{N} \mapsto \mathbb{N}$$

The running time of $\mathcal{M}$ *is bounded by* T *if*

$$\forall w \in \Sigma^*, \forall \pi_w, |\pi_w| \leq T(|w|)$$

Definition 5.8 (Space usage)**.** *The space usage of at configuration* C_i *is*

$$space(C_i) = |w_{out}| + \sum_{j=1}^{k} |w_{wj}|$$

Definition 5.9.

$$S : \mathbb{N} \mapsto \mathbb{N}$$

The space usage of $\mathcal{M}$ *is bounded by* S *if*

$$\forall w \in \Sigma^*, \forall \pi_w, \forall C \in \pi_w, space(C) \leq S(|w|)$$

Definition 5.10. L *is computable by* $\mathcal{M}$ *if* $L = L(\mathcal{M})$

Definition 5.11. L *is computable in time* T *if* $\exists \mathcal{M}$

1. $L = L(\mathcal{M})$ *and*
2. *the running time of* $\mathcal{M}$ *is bounded by* T

Definition 5.12. L *is computable in space* S *if* $\exists \mathcal{M}$

1. $L = L(\mathcal{M})$ *and*
2. *the space usage of* $\mathcal{M}$ *is bounded by* S

Theorem 5.13 (Church-Turing Thesis)**.** *Any model of computation is only as powerful as the Turing machine.*
$\forall L, \exists N$ *in a new model of cmputation,* $L(N) = L \implies \exists \mathcal{M} \in TM, L(\mathcal{M}) = L$

5.4 Complexity classes

Definition 5.14 ($NTIME$)**.**

$$NTIME(T(\cdot)) = \{L \mid L \text{ is computable in time } T(\cdot)\}$$

Definition 5.15 ($DTIME$)**.**

$$DTIME(T(\cdot)) = \{L \mid \exists \text{ deterministic } \mathcal{M}, L(\mathcal{M}) = L, \mathcal{M} \text{ running time is bounded by } T(\cdot)\}$$

Definition 5.16 ($NSPACE$)**.**

$$NSPACE(S(\cdot)) = \{L \mid L \text{ is computable in space } S(\cdot)\}$$

Definition 5.17 ($DSPACE$)**.**

$$DSPACE(S(\cdot)) = \{L \mid \exists \text{ deterministic } \mathcal{M}, L(\mathcal{M}) = L, \mathcal{M} \text{ space usage is bounded by } S(\cdot)\}$$

$$P = PTIME = \bigcup_{k \geq 0} DTIME(f_k), \quad f_k(n) = n^k$$
$$PSPACE = \bigcup_{k \geq 0} DSPACE(f_k), \quad f_k(n) = n^k$$
$$NP = NPTIME = \bigcup_{k \geq 0} NTIME(f_k), \quad f_k(n) = n^k$$
$$NPSPACE = \bigcup_{k \geq 0} NSPACE(f_k), \quad f_k(n) = n^k$$

Theorem 5.18.

$$PTIME \subseteq NPTIME$$

Theorem 5.19.

$$PSPACE \subseteq NPSPACE$$

Theorem 5.20.

$$NPSPACE \subseteq PSPACE$$

Proof. □

Corollary 5.21.

$$PSPACE = NPSPACE$$

5.5 Hardness

Definition 5.22 (Hardness of Turing machine)**.** $\mathcal{M}_1 \leq \mathcal{M}_2$ *if all runs of* $\mathcal{M}_1$ *are smaller than* $\mathcal{M}_2$

5.6 Reduction

Theorem 5.23 (Constant speed up theorem)**.** *L is computable by $f \implies L$ is computable by f', where*

$$f'(n) = f(\frac{n}{c})$$

Theorem 5.24. *For all non-deterministic turing machines $\mathcal{M}_1$ with, there exists a deterministic turing machine $\mathcal{M}_2$, such taht $L(\mathcal{M}_1) = L(\mathcal{M}_2)$*

Lemma 5.25. *For all deterministic turing machine with $\mathcal{M}_1$ with k worktapes, there exists a deterministic turing machine $\mathcal{M}_2$ with 1 worktape such that $L(\mathcal{M}_1) = L(\mathcal{M}_2)$. Same for non deterministic TM.*

Consider deterministic TM $\mathcal{M}$ with an output tape, it induces function

$$f_{\mathcal{M}} : \Sigma^* \mapsto \Gamma^*$$

Such that

$$f_{\mathcal{M}} = \begin{cases} \text{undefined, if } \mathcal{M} \text{ does not terminate;} \\ u, \text{ if } \mathcal{M} \text{ terminates, such that in the final configuration, output tape contains } u \end{cases}$$

Corollary 5.26. *$f_{\mathcal{M}}$ is total if $\mathcal{M}$ is terminating*

Theorem 5.27. *A partial function $f : \Sigma^* \mapsto \Gamma^*$ is computable is $\exists \mathcal{M}$ that induces $f_{\mathcal{M}}, f = f_{\mathcal{M}}$*

Theorem 5.28. *A total function is computable in time $T : \mathbb{N} \mapsto \mathbb{N}$ if there exists $\mathcal{M}$ such that $f_{\mathcal{M}} = f$, and $\mathcal{M}$ is running time bounded by T*

5.7 Cook–Levin Theorem

Theorem 5.29 (Cook–Levin)**.** *SAT is NP-complete. In particular, for every nondeterministic Turing machine $\mathcal{M}$ running in polynomial time, there exists a polynomial-time reduction mapping any input w to a Boolean formula $\Phi_{\mathcal{M},w}$ such that*

$$w \in L(\mathcal{M}) \iff \Phi_{\mathcal{M},w} \text{ is satisfiable.}$$

Proof Outline.

1. SAT $\in$ NP.
2. Every language in NP reduces to SAT.

We focus on (2) under the Turing machine semantics defined above.

Step 1: SAT $\in$ NP

Given a Boolean formula φ, guess a valuation and evaluate φ in time polynomial in $|\varphi|$. Hence SAT $\in$ NP.

Step 2: Reduction from NP to SAT

Let $L \in$ NP. Then there exists a nondeterministic Turing machine $\mathcal{M}$ and a polynomial $p(n)$ such that for every input w:

$$w \in L \iff \mathcal{M} \text{ has an accepting run on } w \text{ of length } \leq p(|w|).$$

Fix w with $|w| = n$, and let $T = p(n)$. We construct a Boolean formula $\Phi_{\mathcal{M},w}$ expressing:

"There exists an accepting run of $\mathcal{M}$ on w of length $\leq T$."

Encoding Configurations

Since $\mathcal{M}$ runs for at most T steps:

- The input head moves at most T cells.
- Each work tape head moves at most T cells.

Thus each tape region used is bounded by $O(T)$ cells.
We represent the entire computation as a *tableau*:

$$\text{Time } 0, 1, \ldots, T \quad \times \quad \text{Tape positions } -T, \ldots, T$$

Each row corresponds to a configuration.

Propositional Variables

We introduce variables:

- $Q_{t,q}$: machine is in state q at time t.
- $H^i_{t,x}$: head of tape i is at position x at time t.
- $S^i_{t,x,\gamma}$: tape i at position x contains symbol γ at time t.

Here $i = 0$ denotes the input tape and $i = 1, \ldots, k$ the work tapes.
The number of variables is polynomial in T.

Constraints

The formula $\Phi_{\mathcal{M},w}$ is a conjunction of four parts.

(A) Exactly-One Constraints For every time t:

- Exactly one state holds:

$$\bigvee_{q \in Q} Q_{t,q} \quad \wedge \quad \bigwedge_{q \neq q'} (\neg Q_{t,q} \vee \neg Q_{t,q'})$$

- Exactly one head position per tape.
- Exactly one symbol per tape cell.

(B) Initial Configuration We enforce:

- Q_{0,q_0}.
- Input tape contains w.
- Work tapes contain blanks.
- All heads at starting position.

All of these are expressed as unit clauses.

(C) Transition Constraints For every $t < T$ and every possible local configuration, we encode the transition relation.

Fix a transition

$$\tau = \langle q, \sigma, \gamma_1, \ldots, \gamma_k, q', d, \gamma'_1, \ldots, \gamma'_k, d_1, \ldots, d_k \rangle.$$

For each position x, we enforce:
If at time t:

- state is q,
- input head at x reading σ,
- work head i at x_i reading γ_i,

then at time $t+1$:

- state is q',
- written symbols updated,
- heads moved according to d, d_i,
- all other cells unchanged.

Each such condition is encoded by clauses of the form:

$$\text{(current configuration)} \rightarrow \text{(next configuration)}$$

which is translated into CNF using Tseitin transformation.

(D) Acceptance We require that for some $t \leq T$:

$$Q_{t,q_{Acc}}$$

This is encoded as:

$$\bigvee_{t=0}^{T} Q_{t,q_{Acc}}.$$

Correctness

Soundness If $\mathcal{M}$ has an accepting run of length $\leq T$, assign variables according to that run. All constraints (A)–(D) are satisfied. Hence $\Phi_{\mathcal{M},w}$ is satisfiable.

Completeness If $\Phi_{\mathcal{M},w}$ is satisfiable, the satisfying assignment defines:

- A unique state per time,
- A unique head position per time,
- A unique symbol per cell,

and transition constraints ensure each configuration is a valid successor of the previous one. Thus we extract a valid accepting run of $\mathcal{M}$ on w. Hence $w \in L(\mathcal{M})$.

Size and Complexity

- Time bound $T = p(n)$.
- Tableau size $O(T^2)$.
- Number of variables and clauses polynomial in T.

Therefore $\Phi_{\mathcal{M},w}$ can be constructed in polynomial time.

Conclusion

We have shown:

$$L \in \mathsf{NP} \quad \Longrightarrow \quad L \leq_p \text{SAT}.$$

Since SAT $\in \mathsf{NP}$, we conclude:

$$\text{SAT is NP-complete.}$$

□

6 Syntax of First Order Logics

Let consider the limitation of propositional logics. Propositional logics intuitively represents choices, and each proposition has no internal structure, it is simply a boolean value to be evaluated. And since formula needs to be finitely long, we cannot express problems of infinite size. Thus we introduce first order logic, which has constant, variables and quantifiers, allowing us to express more nuanced ideas.

6.1 Vocabulary of FOL

1. Signature: $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R})$
   - Constant symbols: $\mathcal{C}$
   - Function symbols: $\mathcal{F}$
   - Relation symbols: $\mathcal{R}$
2. Connector: $C = \{\neg, \vee\}$
3. Variables: V
4. Quantifiers: $\{\forall, \exists\}$
5. Equality: $=$ as a special relation

Definition 6.1. *Term Similar to how we define formula in propositonal logic, a term can also be recursively defined as*

$$t := c \in \mathcal{C} \mid x \in V \mid f \underbrace{(t, \dots)}_{arity(f)}, f \in \mathcal{F}$$

Here we use term to represent the idea of an "atom" in FOL.

Definition 6.2. *Formula*

$$\phi := t = t \mid R \underbrace{(t, \dots)}_{arity(R)}, R \in \mathcal{R} \mid \phi \vee \phi \mid \neg\phi \mid \exists x \in V, \phi \mid \forall x \in V, \phi$$

6.2 Modellng with FOL

Example 6.3. *Number line*

$$\Sigma = (\{0\}, \{succ\}, \{<\})$$

Under this signature, we can write formula:

- $\phi_1 = \forall x \exists y, y = succ(x)$
- $x = 0$
- $0 = 1$

These are all syntactically valid formula.

7 Sematics of First Order Logics

In propositional logics, sematics are conveyed by valuations, which assigns true of false to each proosition. In FOL, we need to define the universe and interpret the symbols to unpack the sematics.

Definition 7.1. *Structure*

$$\mathfrak{A} = (U, I)$$

A structure is a tuple of U, an non-empty universe set, and I, the interpretation of symbols.

Definition 7.2. *Interpretation An interpretation is a mapper from symbols to elements in the universe.*

- *For every constant* $c \in \mathcal{C}, I(c) \in U$
- *For every function symbol* $f \in \mathcal{F}, I(f) \in [U^k \mapsto U], k = arity(f)$
- *For every relation symbol* $R \in \mathcal{R}, I(R) \subseteq U^k, k = arity(R)$

Definition 7.3. *Assignment Given a strucutre* $\mathfrak{A} = (U, I)$

$$\alpha \in [V \mapsto U]$$

Is an assignment of each variable to elements in the universe.

Definition 7.4. *Valuation of terms*

$$Val_{\mathfrak{A},\alpha}(c) = I(c) \in U$$
$$Val_{\mathfrak{A},\alpha}(x) = \alpha(c) \in U$$
$$Val_{\mathfrak{A},\alpha}(f(t,\dots)) = I(f)(Val_{\mathfrak{A},\alpha}(t),\dots) \in U$$

Definition 7.5. *Substitution*

$$\alpha[x \mapsto u](z) = \begin{cases} u, & \text{if } z = u \\ \alpha(z), & \text{otherwise} \end{cases}$$

Definition 7.6. *Valuation of Formula*

Base case: $\phi \equiv t_1 = t_2$

$$\mathfrak{A}, \alpha \models \phi \iff Val_{\mathfrak{A},\alpha}(t_1) = Val_{\mathfrak{A},\alpha}(t_2)$$

Inductive cases:

1. $\phi \equiv R \underbrace{(t,\dots)}_{arity(R)}$

$$\mathfrak{A}, \alpha \models \phi \iff \underbrace{(Val_{\mathfrak{A},\alpha}(t),\dots)}_{arity(R)} \in I(R)$$

2. $\phi \equiv \neg\varphi$

$$\mathfrak{A}, \alpha \models \phi \iff \mathfrak{A}, \alpha \not\models \varphi$$

3. $\phi \equiv \phi_1 \vee \phi_2$

$$\mathfrak{A}, \alpha \models \phi \iff \mathfrak{A}, \alpha \models \phi_1 \ \ OR \ \mathfrak{A}, \alpha \models \phi_2$$

4. $\phi \equiv \exists x, \varphi$

$$\mathfrak{A}, \alpha \models \phi \iff \text{there is } u \in U, \mathfrak{A}, \alpha[x \mapsto u] \models \phi$$

5. $\phi \equiv \forall x, \varphi$

$$\mathfrak{A}, \alpha \models \phi \iff \text{for all } u \in U, \mathfrak{A}, \alpha[x \mapsto u] \models \phi$$

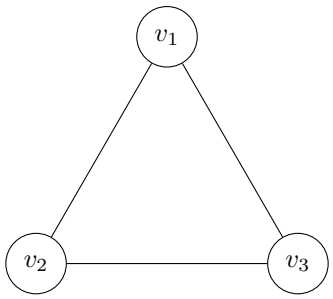
Example 7.7. *graphs Consider signature*

$$\Sigma = (\{s, t, u\}, \emptyset, \{E\})$$

We can come up with a structure

$$\begin{aligned}\mathfrak{A} =&(U_G, IG)\\ U_G =&\{v_1, v_2, v_3\}\\ I_G(s) =& v_1\\ I_G(t) =& v_2\\ I_G(u) =& v_3\\ I_G(E) =& \{(v_1, v_2), (v_2, v_3), (v_3, v_1)\}\end{aligned}$$

The graph represented by $\mathfrak{A}$ is:



Definition 7.8. *Variables and Free variables Variables in a term is defined as:*

$$\begin{aligned}Var(x) &= x\\ Var(c) &= \emptyset\\ Var(f(t_1, \ldots, t_k)) &= \bigcup_1^k Var(t_i)\end{aligned}$$

Free variables in a formula can be defined as:

$$FVar(\forall x, \phi) = FVar(\phi) \setminus \{x\}$$

$$FVar(\exists x, \phi) = FVar(\phi) \setminus \{x\}$$

In words, free variables are variables apppear in a formula that are not bounded by quantifers

Definition 7.9. *Satisfaction of FOL formula*

$$\phi \text{ is satisfiable} \iff \exists(\mathfrak{A}, \alpha), \mathfrak{A}, \alpha \models \phi$$

Definition 7.10. *Logical Consequence*

$$\Gamma \models \phi \iff \forall(\mathfrak{A}, \alpha), \mathfrak{A}, \alpha \models \Gamma \implies \mathfrak{A}, \alpha \models \phi$$

Lemma 7.11 (Relevance Lemma for FOL)**.**

8 Compactness of First Order Logics

Definition 8.1. *Compactness Theorem for First Order Logics*

$$\Gamma \text{ is satisfiable} \iff \Gamma \text{ is finitely satisfiable}$$

$$\Gamma \text{ is satisfiable} \iff \forall \Gamma' \subseteq_{fin} \Gamma, \exists(\mathfrak{A}, \alpha), \mathfrak{A}, \alpha \models \Gamma'$$

Theorem 8.2. *Structures are not countable, and the uncountability is uncapped, as U and I can be arbitrarily defined*

Definition 8.3. *Prenex Normal Form ϕ is in Prenex Normal Form if all quantifiers are in the beginning of the formula.*

$$\phi \equiv Q_1 x_1 Q_2 x_2 \dots \psi$$

Where ψ is the matrix of ϕ, which is a quantifier-free formula.

8.1 Prenex Normal Form

1. **Remove implication and biconditional.**

   Replace

   $$A \to B \equiv \neg A \lor B$$

   $$A \leftrightarrow B \equiv (A \to B) \land (B \to A)$$

2. **Push negations inward (Negation Normal Form).**

   Apply De Morgan's laws and quantifier negation:

   $$\neg(A \land B) \equiv \neg A \lor \neg B$$

   $$\neg(A \lor B) \equiv \neg A \land \neg B$$

   $$\neg \forall x \, \varphi \equiv \exists x \, \neg \varphi$$

   $$\neg \exists x \, \varphi \equiv \forall x \, \neg \varphi$$

   After this step, negations appear only directly in front of atomic formulas.

3. **Standardize variables apart.**

   Rename bound variables so that each quantifier binds a unique variable and no variable is bound by more than one quantifier.

4. **Move quantifiers outward.**

   Move quantifiers across logical connectives when the variable does not occur free in the other formula. For example,

   $$(\forall x \, A) \land B \equiv \forall x (A \land B) \quad \text{if } x \notin \mathrm{FV}(B)$$

   $$(\exists x \, A) \lor B \equiv \exists x (A \lor B) \quad \text{if } x \notin \mathrm{FV}(B)$$

   Continue moving quantifiers outward until all quantifiers appear at the front of the formula.

5. **Obtain Prenex Normal Form.**

   The formula now has the form

   $$Q_1 x_1 \; Q_2 x_2 \; \dots \; Q_n x_n \; \varphi$$

   where each $Q_i \in \{\forall, \exists\}$ and φ is quantifier-free.

Lemma 8.4. *ψ, the PNF of ϕ is logically equivalent to ϕ*

$$\forall(\mathfrak{A}, \alpha), \mathfrak{A}, \alpha \models \psi \iff \mathfrak{A}, \alpha \models \phi$$

Proof. □

Definition 8.5. *Universally quantified formula ϕ is universally wuantified if all quantifiers in ϕ are $\forall$. Analogous to Existantial quantified formula*

Definition 8.6. *Sentence ϕ is a sentence if there is no free variables.*

$$FVar(\phi) = \emptyset$$

Definition 8.7. *Skolemization Transform PNF into universally quantified PNF.*
$\forall \phi, \exists \phi'$, such that

1. *ϕ and ϕ' are equisatisfiable.*
2. *ϕ' is universally quantified PNF*

8.2 Skolemization

1. **Convert the formula to prenex normal form.**

   Transform the formula so that all quantifiers appear at the front:

   $$Q_1 x_1 \, Q_2 x_2 \, \ldots \, Q_n x_n \, \varphi$$

   where φ is quantifier-free.

2. **Process quantifiers from left to right.**

   For each existential quantifier $\exists x$, determine the universal variables that precede it in the prefix.

3. **Replace existential variables with Skolem terms.**

   - If $\exists x$ has no preceding universal quantifiers, replace x with a new Skolem constant c.

     $$\exists x \, P(x) \quad \Rightarrow \quad P(c)$$

   - If $\exists x$ is preceded by universal variables $u_1, \ldots, u_k$, replace x with a new Skolem function $f(u_1, \ldots, u_k)$.

     $$\forall u_1 \ldots \forall u_k \exists x \, P(x, u_1, \ldots, u_k) \;\Rightarrow\; P(f(u_1, \ldots, u_k), u_1, \ldots, u_k)$$

4. **Remove the existential quantifier.**

   After substitution with the Skolem term, delete the corresponding existential quantifier from the prefix.

5. **Repeat until no existential quantifiers remain.**

   The resulting formula contains only universal quantifiers.

6. **Optionally drop universal quantifiers.**

   In many applications (e.g., resolution), the remaining universal quantifiers are omitted, since variables are implicitly assumed to be universally quantified.

Theorem 8.8 (Herbrand's Theorem)**.** *Informally, if a first-order formula ϕ is satisfiable, then its satisfiability is witnessed by a special kind of structure called a* Herbrand structure*.*

Definition 8.9 (Herbrand structure)**.** *Let $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{P})$ be a signature. A structure*

$$\mathfrak{A} = (U, I)$$

is a Herbrand structure *if*

1. *The universe is the* Herbrand universe

   $$U = T_{\mathcal{C} \cup \mathcal{F}},$$

   the set of all ground terms constructed from the constant symbols $\mathcal{C}$ and function symbols $\mathcal{F}$.

2. *The interpretation I is fixed syntactically:*

   - *For every constant symbol $c \in \mathcal{C}$,*

     $$I(c) = c$$

   - *For every function symbol $f \in \mathcal{F}$ of arity n,*

     $$I(f)(t_1, \ldots, t_n) = f(t_1, \ldots, t_n)$$

Definition 8.10. *Ground term*

$$gt := c \mid f(gt, \dots)$$

Basically terms without variables

Definition 8.11. *Ground Atom (Let's ignore = first)*

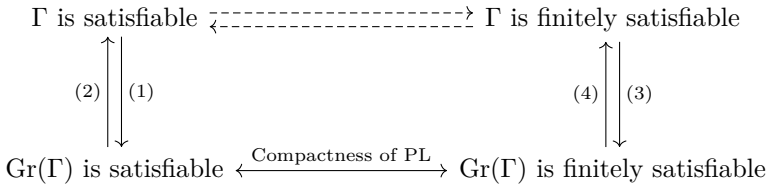
$$ga := R(gt, \dots)$$

Definition 8.12 (Ground Formula)**.**

$$gf := gt = gt \mid R(gt, \dots) \mid \neg gf \mid gf \vee gf$$

No variables, no quantifiers

Now that we have developed the machinery needed to prove Compactness for first-order logic, we outline the overall proof strategy. From the definition of grounding, it is clear that the goal is to connect first-order logic with propositional logic. By grounding, we eliminate variables and quantifiers, and treat each ground atom as a propositional atom. In this way, we translate the satisfiability problem for a first-order theory into a corresponding satisfiability problem in propositional logic. This suggests the following proof plan: establish a correspondence between satisfiability and finite satisfiability for Γ and for its grounding $\mathrm{Gr}(\Gamma)$, then apply Compactness of propositional logic to $\mathrm{Gr}(\Gamma)$ in order to deduce Compactness for first-order logic.



Definition 8.13 (Ground instantiation)**.** *ϕ is a universally quantified sentence in PNF, let ψ be a ground formula, with the same signature. ψ is a ground instantiation of ϕ, or $Gr(\phi) = \psi$ if:*

$$\phi = \forall x_1 \forall x_2, \dots, \forall x_n \cdot \gamma$$

There are n ground terms $gt_1, gt_2, \dots, gt_n$, such that

$$\psi = \gamma[x_1 \mapsto gt_1][x_2 \mapsto gt_2] \dots [x_n \mapsto gt_n]$$

Definition 8.14 (Ground instantiation of a set)**.**

$$Gr(\Gamma) = \{\psi \mid \forall \phi \in \Gamma, \psi = Gr(\phi)\}$$

Lemma 8.15 (Ground instantiation of a formula is countable)**.**

Corollary 8.16 (Ground instantiation of a set is countable)**.**

Definition 8.17 (Ground valuation)**.**

$$gv : gt_\Sigma \mapsto \{true, false\}$$

$$gv \models_{gr} R(t, \dots) \iff gv(R(t, \dots)) = true$$
$$gv \models_{gr} \neg gf \iff gv \not\models_{gr} gf$$
$$gv \models_{gr} gf_1 \vee gf_2 \iff gv \models_{gr} gf_1 \ OR \ gv \models_{gr} gf_2$$

Definition 8.18 (Propositional satisfiable)**.** *A ground formula gf is propositionally satisfiable if*

$$\exists gv \models_{gr} gf$$

8.3 Proof of the compactness theorem for FOL

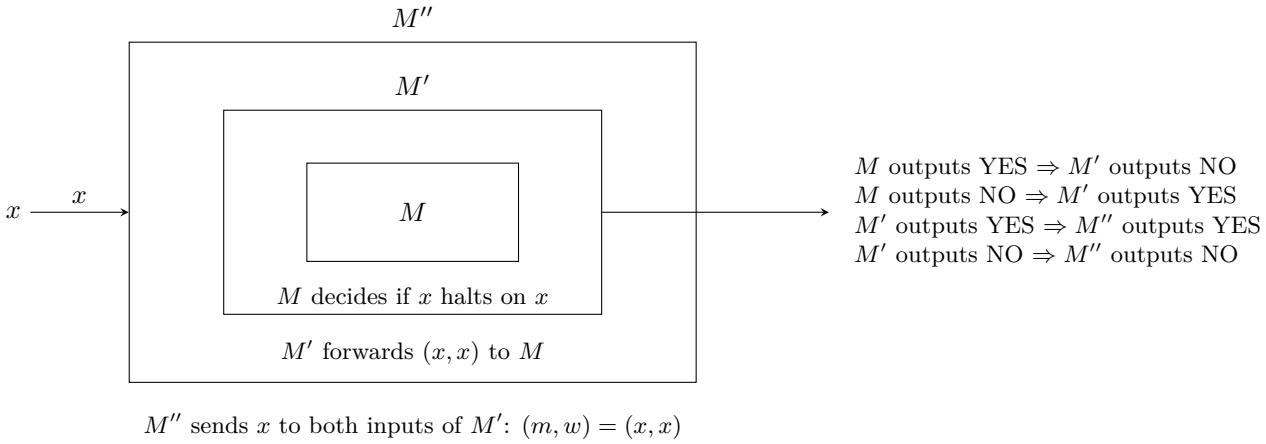
8.3.1 Γ is satisfiable $\implies$ $Gr(\Gamma)$ is satisfiable

9 Computability

9.1 Halting problem

Theorem 9.1. *The halting problem is undecidable*

Proof.



M'' sends x to both inputs of M': $(m, w) = (x, x)$

Now consider $M''(M'')$.

Case 1: $M''(M'') = NO$

□

Definition 9.2 (Recursively Enumerable)**.** *$L \subseteq \Sigma^*$ is recursively enumerable (RE), if there is a Turing machine $\mathcal{M}$, such that $L = L(\mathcal{M})$. Other equivalent terms for an RE language are:*

- *Semi-decidable*
- *Turing recognizable*

Definition 9.3 (Recursive)**.** *$L \subseteq \Sigma^*$ is a recursive(REC) language if there is a Turing machine $\mathcal{M}$, such that $L = L(\mathcal{M})$, and $\mathcal{M}$ is terminating. Recursive languages are also called decidable*

Theorem 9.4.

$$L_H = \{\langle \mathcal{M}, w\rangle \mid w \text{ terminates on } \mathcal{M}\} \in RE$$

Proof. We can construct a TM $\mathcal{M}_{Halt}$ which is the turing machine that returns YES if its input machine terminates on input string.

$\mathcal{M}_{Halt}(\mathcal{M}, w)$:
    $\mathcal{M}(w)$
    `return YES`

This surffice to show that $L_H \in RE$

$L_H \notin REC$

Similar to how we prove undecidability of the Halting problem □

Theorem 9.5 (Membership)**.**

$$MP = \{\langle \mathcal{M}, w\rangle \mid w \text{ is accepted by } \mathcal{M}\} \in RE$$

Proof. We can construct TM

$\mathcal{M}_{UTM}(\mathcal{M}, w)$:
    $b = \mathcal{M}(w)$
    return b

□

Theorem 9.6.

$$L \in REC \iff \bar{L} \in REC$$

Proof. $L \in REC \implies \exists \mathcal{M}, L = L(\mathcal{M}), \mathcal{M}$ is terminating, we can construct another TM

$\mathcal{M}'(w)$:
    $b = \mathcal{M}(w)$
    return $\neg b$

It is easy to show that $L(\mathcal{M}') = \bar{L}$, and $\mathcal{M}'$ is terminating. □

Theorem 9.7.

$$L \in RE \wedge \bar{L} \in RE \implies L \in REC$$

Proof. Consider then TM for L and $\bar{L}$, $\mathcal{M}_1$ and $\mathcal{M}_2$, we can construct a new TM

$\mathcal{M}(w)$:
    for $i > 0$:
        run the i^{th} step of $\mathcal{M}_1$
        if $\mathcal{M}_1$ reaches $q_{\mathcal{M}_1, Acc}$
            return YES
        run the i^{th} step of $\mathcal{M}_2$
        if $\mathcal{M}_2$ reaches $q_{\mathcal{M}_2, Acc}$
            return NO

Since $\forall w, \forall L \subseteq \Sigma^*, w \in L \vee w \in \bar{L}, w$ must be accepted by either $\mathcal{M}_1$ or $\mathcal{M}_2$, therefore $\mathcal{M}$ is terminating. □

9.2 Reduction

Definition 9.8. *$L_1 \leq_m L_2$ if there is a computable function f, such that*

$$w \in L_1 \implies f(w) \in L_2$$
$$w \notin L_1 \implies f(w) \notin L_2$$

Theorem 9.9.

$$L_1 \leq_m L_2, L_2 \in REC \implies L_1 \in REC$$

Theorem 9.10.

$$L_1 \leq_m L_2, L_2 \in RE \implies L_1 \in RE$$

Proof. □

Theorem 9.11.

$$L_1 \leq_m L_2, L_1 \in REC \not\Longrightarrow L_2 \in REC$$

Proof. Consider the following recursive language:

$$L_1 = \{\langle \mathcal{M}, w\rangle \mid w \text{ termintes on } \mathcal{M} \text{ in 10 steps}\}$$

and

$$L_H = \{\langle \mathcal{M}, w\rangle \mid w \text{ termintes on } \mathcal{M}\}$$

We have shown that $L_H \in RE \backslash REC$. We can construct the following terminating TM to define a computable function f

---

$\mathcal{M}_f(\langle \mathcal{M}, w\rangle)$:
    run $\mathcal{M}_1$ on w for 10 steps
    if output is YES:
        return $\langle \mathcal{M}, w\rangle$
    else:
        return $\langle NH, w\rangle$

Where NH is the encoding of a TM that loops forever on all input.
It is easy to show that

$$\langle \mathcal{M}, w\rangle \in L_1 \implies f(\langle \mathcal{M}, w\rangle) \in L_H$$
$$\langle \mathcal{M}, w\rangle \notin L_1 \implies f(\langle \mathcal{M}, w\rangle) \notin L_H$$

But $L_H \notin REC$ as we have previously shown. Contradiction □

9.3 RE-hardness

There are languages beyond the RE class. For example

Theorem 9.12.

$$\bar{L}_H \notin RE$$

Proof. Suppose otherwise, $\bar{L}_H \in RE, L_H \in RE \implies L_H \in REC$, contradiction □

Definition 9.13 (RE-hard)**.** *L is RE-hard, if*

$$\forall L' \in RE, L' \leq_m L$$

9.4 Validity of FOL

Definition 9.14.

$$VALID : \{\varphi \mid \varphi \text{ is valid } \}$$

Theorem 9.15. $VALID \in RE$

Theorem 9.16 (Church-Turing Theorem)**.** *$VALID$ is RE-hard*

HW1

Definition 9.17 (Generic Closure)**.** *Let U be a universe of elements and let O be a set of functions whose domain is U and codomain is $\mathscr{P}(U)$. The arity of a function $f \in O$ is denoted by $arity(f)$. A function f of arity r has type*

$$f : U^r \to \mathscr{P}(U).$$

*Let $I \subseteq U$ be an initial set. A set $S \subseteq U$ is said to be (I, O)-*closed *if:*

1. *$I \subseteq S$.*
2. *For every $f \in O$ with $arity(f) = r$ and for all $e_1, \ldots, e_r \in U$, if $e_1, \ldots, e_r \in S$, then $f(e_1, \ldots, e_r) \subseteq S$.*

Lemma 9.18. *Let U be a universe, O a set of operators, and $I \subseteq U$ an initial set. There exists a smallest (I, O)-closed set.*

Define $\mathcal{P}(U)$ as a collection of all ordering of all elemenst in the power set $\mathscr{P}(U)$

Q1

Claim: The smallest (I, O)-closed set can be constructed by taking the intersection of all (I,O)-closed set, ie.

$$AIOCS = \{S \subseteq U \mid S \text{ S is (I,O)-closed}\}$$

$$T_0 = \bigcap AIOCS$$

Existence

First we prove the existence of such a set.

Proof. Suppose otherwise,

$$\begin{aligned}
&T_0 = \bigcap AIOCS = \emptyset \\
&T_0 = \{x \in \mathcal{P}(U) \mid \forall S \in AIOCS, x \in S\} \\
&T_0 = \emptyset \to x \in S \text{ is vacuously true } \forall x \in \mathcal{P}(U) \to T_0 = \mathcal{P}(U)
\end{aligned}$$

contradiction. □

Closed

Now we prove that T_0 is (I,O)-closed.

Proof. Property 1:

$$\begin{aligned}
&\forall S \in AIOCS, I \subseteq S \\
&\quad \to \forall i \in I, \forall S \in AIOCS, i \in S \\
&\quad \to \forall i \in I, i \in \bigcap AIOCS \\
&\quad \to \forall i \in I, i \in T_0
\end{aligned}$$

Property 2:

$$\begin{aligned}
&\forall S \in AIOCS, T_0 \subseteq S \\
&\quad \text{Let } e_1, e_2, \ldots, e_r \in T_0, \forall S \in AIOCS, e_1, e_2, \ldots, e_r \in S \\
&\quad \to \exists f \in \mathcal{O}, f(e_1, e_2, \ldots, e_r) \in S \\
&\quad \to \forall e_1, e_2, \ldots, e_r \in T_0, \exists f \in \mathcal{O}, f(e_1, e_2, \ldots, e_r) \in \bigcap AIOCS \\
&\quad \to f(e_1, e_2, \ldots, e_r) \in T_0
\end{aligned}$$

□

Uniqueness

Claim:

$$\forall T_0' \in \{T \in \mathcal{P}(U) \mid \forall S \in AIOCS, T \subseteq S\}, T_0' = T_0$$

Proof. Suppose otherwise, $\exists T_0' \in AIOCS, T_0' \neq T_0$.

$$\begin{aligned} T_0, T_0' \in AIOCS &\rightarrow \forall S \in AIOCS, T_0 \subseteq S, T_0' \subseteq S \\ &\rightarrow T_0 \subseteq T_0', T_0' \subseteq T_0 \\ &\rightarrow T_0 = T_0' \end{aligned}$$

□

Q2

For each of the following strings, prove or disprove that it is a well-formed formula:

1. $w_1 = \epsilon$ (the empty string).
2. $w_2 = (\neg p)$ where $p \in P$.
3. $w_3 = (p\neg)$ where $p \in P$.

2.1

$$w_1 = \epsilon \notin FORM$$

Claim: $\forall w \in FORM, |w| > 0$ I'd like to use definition 2.19 and lemma 2.21 which I proved later without using the conslusion from this question.

$$FORM = INDFORM = \bigcup_{i \geq 0} FORM_i$$

Proof. Base case:

$$\begin{aligned} &FORM_0 = P \\ &\forall \phi \in FORM_0, \phi = p \in P \\ &\rightarrow \forall \phi \in FORM_i, |\phi| > 0 \end{aligned}$$

Inductive Hypothesis: Suppose

$$\forall i \geq 0, \forall \phi \in FORM_i, |\phi| > 0$$

$$\begin{aligned} &FORM_{i+1} = FORM_i \cup \{(\neg\phi) \mid \phi \in FORM_i\} \cup \{(\phi_1 \vee \phi_2) \mid \phi_1, \phi_2 \in FORM_i\} \\ &\forall \psi \in FORM_{i+1}, \exists \phi \in FORM_i, |\psi| = |\phi| \text{ or } |\psi| = |\phi| + 3 \text{ or} \\ &\exists \phi_1, \phi_2 \in FORM_i, |\psi| = |\phi_1| + |\phi_2| + 3 \end{aligned}$$

□

2.2

$$p \in P \rightarrow p \in FORM \rightarrow (\neg p) \in FORM$$

2.3

$(p\neg) \notin FORM$

Proof. Suppose otherwise, $(p\neg\epsilon) \in FORM$

$$\begin{aligned} \forall \phi \in FORM, (\neg\phi) \in FORM \\ \rightarrow \epsilon \in FORM \end{aligned}$$

Contradiction from 2.1 □

Q3

Fix $U = \mathbb{R}$. For each of the following sets S, define a finite initial set $I \subseteq \mathbb{R}$ and a set of operators O with $|O| < 10$ such that the smallest (I, O)-closed set is S.

1. $S = \{0, 1, 2, 3, \dots\}$.
2. $S = \{1, 3, 5, 7, \dots\}$.
3. $S = \mathbb{Q}$.
4. $S = \{\frac{1}{3}, \frac{2}{3}, \frac{4}{3}, \frac{5}{3}, \frac{7}{3}, \dots\}$.

Answer:

1. $\mathbb{N} : I = \{0\}, O = \{f(x) = x + 1\}$
2. $Odd : I = \{1\}, O = \{f(x) = x + 2\}$
3. $\mathbb{Q} : I = \{0\}, O = \{f(x) = -x, f(x) = x + 1, f(x, y) = \begin{cases} \frac{x}{y} & \text{if } y \neq 0 \\ \emptyset & \text{if } y = 0 \end{cases}\}$
4. $\{\frac{x}{3} \mid x \in \mathbb{N}\} \setminus \mathbb{N} : I = \{\frac{1}{3}, \frac{2}{3}\}, O = \{f(x) = x + 1\}$

Finite Directed Graphs

Definition 9.19 (Graph Reachability)**.** *Let $G = (V, E)$ be a directed graph. A path from s to t is a sequence of vertices $\pi = u_1, \dots, u_k$ such that:*

1. *$u_1 = s$,*
2. *$(u_i, u_{i+1}) \in E$ for all $1 \leq i < k$,*
3. *$u_k = t$.*

The length of π is $k - 1$. We say that t is reachable from s if $s = t$ or such a path exists. The set of ancestors of t is defined as

$$Reach_t = \{v \in V \mid t \text{ is reachable from } v\}.$$

Q4

Let $G = (V, E)$ be a directed graph and let $t \in V$. Fix the universe $U = V$.

1. Define an initial set I (with $|I| \leq 2$) and a unary operator f such that the smallest $(I, \{f\})$-closed set is the set of vertices reachable from t.
2. Let $d \in \mathbb{N} \setminus \{0\}$ and $r < d$. Define I and a unary operator f such that the smallest $(I, \{f\})$-closed set is the set of vertices reachable from s via paths of length ℓ with $\ell \equiv r \pmod{d}$.

4.1

$I = \{t\}, O = \{f(v) = succ(v)\}$

4.2

$I = \{s\} \cup \{v \in V \mid v \in succ^r(s)\}, O = \{f(v) = succ^d(v)\}$

Q5

Let $G = (V, E)$ be a directed graph and fix $U = V \times V$. Define an initial set I with $|I| \leq |V|$ and a binary operator f such that the smallest $(I, \{f\})$-closed set is

$$AllReachPairs = \{(u, v) \mid v \text{ is reachable from } u\}.$$

5.1

$$I = \{(v, v) \mid v \in V\}, O = \{f((v_1, v_2), (v_3, v_4)) \mapsto \begin{cases} \{(v_1, v_4)\} & \text{if } v_2 = v_3 \\ \emptyset & \text{if } v_2 \neq v_3 \end{cases}\}$$

Inductive Definitions

Definition 9.20 (Inductive Well-Formed Formulae)**.** *Let P be a set of propositions and $C = \{\neg, \vee\}$. Define sets $FORM_i$ as follows:*

$$FORM_0 = P,$$
$$FORM_{i+1} = FORM_i \cup \{(\neg w) \mid w \in FORM_i\} \cup \{(w_1 \vee w_2) \mid w_1, w_2 \in FORM_i\}.$$

Define

$$INDFORM = \bigcup_{i \geq 0} FORM_i.$$

Q6

1. Show that $FORM_i \subseteq FORM_{i+1}$ for all i.
2. Prove or disprove that $FORM_i = FORM_{i+1}$ for some i.
3. Prove or disprove that $FORM_i$ is (P, C)-closed for all i.
4. Prove or disprove that $FORM_i$ is (P, C)-closed for some i.

6.1

Base case: $FORM_0 \subseteq FORM_1$ as

$$FORM_1 = FORM_0 \cup \{(\neg w) \mid w \in FORM_0\} \cup \{(w_1 \vee w_2) \mid w_1, w_2 \in FORM_0\}$$

Wait this is just trivial

6.2

Claim:

$$\exists i \geq 0, FORM_i = FORM_{i+1}$$

It is obvious that $\forall i \geq 0, FORM_i$ is finite. Suppose

6.3

Claim:

$$\forall i \geq 0, FORM_i \text{ is (P,C)-closed}$$

Proof. Suppose otherwise, consider $FORM_0$,

$$FORM_0 = P$$
$$\forall p \in P, (\neg p) \in FORM_0$$

Contradiction □

6.4

Claim:

$$\exists i \geq 0, FORM_i \text{ is (P,C)-closed}$$

Lemma 9.21.

$$\forall i \geq 0, \forall \phi \in FORM_i, |\phi| \leq 2^{i+3} - 3$$

The longest formula in $FORM_i$ has size $2^{i+3} - 3$. Parentheses are counted in string length

Proof. Base case:

$$FORM_0 = P$$
$$\forall \phi \in FORM_0, |\phi| = 1$$

Inductive hypothesis:

$$\forall i \geq 0, \forall \phi \in FORM_i, |\phi| \leq 2^{i+3} - 3$$

Consider all new strings formed from the longest string $\phi \in FORM_i$,

$$|\phi| = 2^{i+3} - 3$$
$$|(\neg\phi)| = 2^{i+3}$$
$$|(\phi \vee \phi)| = 2 \times (2^{i+3} - 3) + 3 = 2^{(i+1)+3} - 3$$

□

Proof. Suppose $\exists i \geq 0, FORM_i$ is (P,C)-closed, and from lemma 2.20 the longest formula in $FORM_i$ has size $|\phi_{max}| = 2^{i+3} - 3$. $\rightarrow \forall \phi \in FORM_i, (\neg\phi) \in FORM_i \rightarrow \phi'_{max} = (\neg\phi_{max}) \in FORM_i, |\phi'_{max}| > 2^{i+3} - 3$, a contradiction.

□

Lemma 9.22. $FORM = INDFORM$.

Q7

Prove the lemma by showing:

1. $INDFORM$ is (P, C)-closed.
2. For every (P, C)-closed set S, $INDFORM \subseteq S$.

7.1

Claim: $INDFORM$ is (P,C)-closed

Proof. **Claim:** $P \subseteq INDFORM$

First condition of (P,C)-closed set:

$$FORM_0 \subseteq INDFORM \rightarrow P \subseteq INDFORM$$

Claim: $\forall w \in INDFORM, (\neg w) \in INDFORM$

$$\begin{aligned} INDFORM = \bigcup_{i \geq 0} FORM_i \\ \rightarrow \forall i \geq 0, FORM_i, FORM_{i+1} \in INDFORM \\ \forall w \in INDFORM, \exists j \geq 0, w \in FORM_j \\ \rightarrow (\neg w) \in FORM_{j+1} \rightarrow (\neg w) \in INDFORM \end{aligned}$$

Claim: $\forall w_1, w_2 \in INDFORM, (w_1 \vee w_2) \in INDFORM$

Case 1: $\exists i \geq 0, w_1, w_2 \in FORM_i$

$$w_1, w_2 \in FORM_i \rightarrow (w_1 \vee w_2) \in FORM_{i+1} \rightarrow (w_1 \vee w_2) \in INDFORM$$

Case 2: $\nexists i \geq 0, w_1, w_2 \in FORM_i$. WLOG, suppose $w_1 \in FORM_i, w_2 \in FORM_j, i < j$

$$\forall w \in FORM_n, w \in FORM_{n+1} \text{By induction } w_1 \in FORM_i \rightarrow w_1, w_2 \in FORM_j$$

Back to case 1.

□

7.2

$$\forall \text{(P,C)-closed set } S, INDFORM \subseteq S$$

To prove this, we only need to to prove that $INDFORM \subseteq S_{min} = \bigcap_{S \text{ is (P,C)-closed}} S$.

Proof. Base case:

$$\bigcup_{i=0}^{0} FORM_i = FORM_0 = P \subseteq S_{min}$$

Inductive hypothesis: Suppose

$$\bigcup_{i=0}^{n} FROM_i \subseteq S_{min}$$

$$\begin{aligned}
&\bigcup_{i=0}^{n} FROM_i \subseteq S_{min} \rightarrow FORM_n \in S_{min} \\
&\quad S_{min} \text{ is (P,C)-closed} \rightarrow \forall i \in [1, n], \forall \phi \in FORM_n, (\neg\phi) \in S_{min}, \\
&\quad \forall i \in [0, n], \forall \phi_1, \phi_2 \in FORM_i, (\phi_1 \vee \phi_2) \in S_{min} \\
&\rightarrow \bigcup_{i=0}^{n} FROM_i \cup \{(\neg\phi) \mid \forall \phi \in \bigcup_{i=0}^{n} FROM_i\} \cup \{(\phi_1 \vee \phi_2) \mid \forall \phi_1, \phi_2 \in \bigcup_{i=0}^{n} FROM_i\} \subseteq S_{min} \\
&\rightarrow \bigcup_{i=0}^{n+1} FROM_i \subseteq S_{min}
\end{aligned}$$

□

Definition 9.23 (Alternative Inductive Definition)**.** *Define* $AltFORM_0 = P$ *and for* $i \geq 0$*:*

$$\begin{aligned}
AltFORM_{i+1} = &\{(\neg w) \mid w \in AltFORM_i\} \\
&\cup \{(w_1 \vee w_2) \mid w_1 \in AltFORM_i,\ \exists j \leq i,\ w_2 \in AltFORM_j\} \\
&\cup \{(w_1 \vee w_2) \mid w_2 \in AltFORM_i,\ \exists j \leq i,\ w_1 \in AltFORM_j\}.
\end{aligned}$$

Define

$$AltINDFORM = \bigcup_{i \geq 0} AltFORM_i.$$

Question 8

1. Show that $FORM_i = \bigcup_{j=0}^{i} AltFORM_j$ for all i.
2. Show that $INDFORM = AltINDFORM$.

8.1

Lemma 9.24.

Proof. Base case:

$$FORM_0 = \bigcup_{j=0}^{0} AltFORM_j$$

$$FORM_0 = P = AltFORM_0$$

Inductive hypothesis: Suppose

$$\forall j \in [0, i], FORM_i = \bigcup_{k=0}^{i} AltFORM_k$$

$$FORM_{i+1} = \bigcup_{k=0}^{i} AltFORM_k \cup \{(\neg\phi) \mid \phi \in \bigcup_{k=0}^{i} AltFORM_k\} \cup \{(\phi_1 \vee \phi_2) \mid \phi_1, \phi_2 \in \bigcup_{k=0}^{i} AltFORM_k\}$$
$$\supseteq \bigcup_{k=0}^{i} AltFORM_k \cup \{(\neg\phi) \mid \phi \in AltFORM_i\} \cup \{(\phi_1 \vee \phi_2) \mid \phi_1, \phi_2 \in \bigcup_{k=0}^{i} AltFORM_k\}$$

$$\bigcup_{k=0}^{i+1} AltFORM_k \subseteq FORM_{i+1}$$

Now we prove the other way:

$$\forall \phi \in FORM_{i+1}, \phi \in \bigcup_{k=0}^{i} AltFORM_k \text{ or } \phi = (\neg\psi), \psi \in \bigcup_{k=0}^{i} AltFORM_k \text{ or } \phi = (\psi_1 \vee \psi_2), \psi_1, \psi_2 \in \bigcup_{k=0}^{i} AltFORM_k$$

Case 1:

$$\phi \in \bigcup_{k=0}^{i} AltFORM_k \rightarrow \phi \in \bigcup_{k=0}^{i+1} AltFORM_k$$

Case 2:

$$\psi \in \bigcup_{k=0}^{i} AltFORM_k \rightarrow \exists j \leq i, \psi \in AltFORM_j$$
$$\phi = (\neg\psi), \psi \in AltFORM_j, \phi \in AltFORM_{j+1}$$
$$\rightarrow \phi \in \bigcup_{k=0}^{i+1} AltFORM_k$$

Case 3:

$$\psi_1, \psi_2 \in \bigcup_{k=0}^{i} AltFORM_k \rightarrow \exists j, l \leq i, \psi_1 \in AltFORM_j, \psi_2 \in AltFORM_l$$
$$\phi = (\psi_1 \vee \psi_2), \psi_1 \in AltFORM_j, \psi_2 \in AltFORM_l, \phi \in AltFORM_{j+1}$$
$$\rightarrow \phi \in \bigcup_{k=0}^{i+1} AltFORM_k$$

$$FORM_{i+1} \subseteq \bigcup_{k=0}^{i+1} AltFORM_k \rightarrow FORM_{i+1} = \bigcup_{k=0}^{i+1} AltFORM_k$$

□

Formation Sequences

Definition 9.25 (Formation Sequence)**.** *A (P, C)-formation sequence is a nonempty finite sequence $\rho = w_1, \ldots, w_k$ such that for each i:*

- $w_i \in P$, *or*
- $w_i = (\neg w_j)$ *for some* $j < i$, *or*
- $w_i = (w_j \vee w_k)$ *for some* $j, k < i$.

The sequence belongs to w if $w_k = w$.

Q9

Give two distinct formation sequences that belong to

$$w = ((\neg p_1) \vee ((\neg p_3) \vee (\neg(p_2 \vee (\neg p_1)))))$$

Two distinct formation sequences ρ_1, ρ_2 are:

ρ_1**:**

$$\begin{aligned}
w_1 &= p_1 \\
w_2 &= (\neg w_1) \\
w_3 &= p_2 \\
w_4 &= (w_3 \vee w_2) \\
w_5 &= (\neg w_4) \\
w_6 &= (p_3) \\
w_7 &= (\neg w_6) \\
w_8 &= (w_7 \vee w_5) \\
w_9 &= (w_2 \vee w_8)
\end{aligned}$$

ρ_2**:**

$$\begin{aligned}
w_1 &= p_1 \\
w_2 &= p_2 \\
w_3 &= p_3 \\
w_4 &= (\neg w_3) \\
w_5 &= (\neg w_1) \\
w_6 &= (w_2 \vee w_5) \\
w_7 &= (\neg w_6) \\
w_8 &= (w_4 \vee w_7) \\
w_9 &= (w_5 \vee w_8)
\end{aligned}$$

These are distinct because the components at index $i = 2$ differ ($(\neg p_1)$ vs p_2).

Lemma 9.26. *$INDFORM = \{w \in \Sigma^* \mid w$ is formation-sequence-well-formed$\}$.*

Q10

Prove the lemma by showing:

1. Every formation-sequence-well-formed string is in $INDFORM$.

2. Every $w \in INDFORM$ has a formation sequence.

Before attemping Q10 from the above definition of formation-sequnce-well-formed, I will prove the following lemma using the definition below first:

Definition 9.27. *Define* $ExFORM_0 = P$

$$\begin{aligned} ExFORM_{i+1} = (&\{(\neg w) \mid w \in ExFORM_i\} \\ &\cup \{(w_1 \vee w_2) \mid w_1 \in ExFORM_i, \exists j \leq i, w_2 \in ExFORM_j\} \\ &\cup \{(w_1 \vee w_2) \mid w_2 \in ExFORM_i, \exists j \leq i, w_1 \in ExFORM_j\} \\ ) &\setminus \bigcup_{j=0}^{i} ExFORM_j \end{aligned}$$

Lemma 9.28.

$$\bigcup_{j=0}^{i} ExFORM_j = FORM_i$$

Proof. Base case:

$$ExFORM_0 = P = FORM_0$$

Inductive Hypothesis: Suppose $\forall n \geq 0$,

$$\bigcup_{i=0}^{n} ExFORM_i = FORM_n$$

From definition 2.19,

$$\begin{aligned} FORM_{n+1} = &\bigcup_{i=0}^{n} ExFORM_i \cup \{(\neg w) \mid w \in \bigcup_{i=0}^{n} ExFORM_i\} \\ &\cup \{(w_1 \vee w_2) \mid w_1 \in \bigcup_{i=0}^{n} ExFORM_i, \exists j \leq i, w_2 \in \bigcup_{i=0}^{n} ExFORM_j\} \\ &\cup \{(w_1 \vee w_2) \mid w_2 \in \bigcup_{i=0}^{n} ExFORM_i, \exists j \leq i, w_1 \in \bigcup_{i=0}^{n} ExFORM_j\} \end{aligned}$$

Definition 9.29.

$$\begin{aligned} FromFORM : n \mapsto &\{(\neg w) \mid w \in \bigcup_{i=0}^{n} ExFORM_i\} \\ &\cup \{(w_1 \vee w_2) \mid w_1 \in \bigcup_{i=0}^{n} ExFORM_i, \exists j \leq i, w_2 \in \bigcup_{i=0}^{n} ExFORM_j\} \\ &\cup \{(w_1 \vee w_2) \mid w_2 \in \bigcup_{i=0}^{n} ExFORM_i, \exists j \leq i, w_1 \in \bigcup_{i=0}^{n} ExFORM_j\} \end{aligned}$$

Lemma 9.30.

$$FORM_{n+1} = \bigcup_{i=0}^{n} ExFORM_i \cup FromFORM(n)$$

Lemma 9.31.

$$ExFORM_{n+1} = (\bigcup_{i=0}^{n} ExFORM_i \cup FromFORM(n)) \setminus \bigcup_{i=0}^{n} ExFORM_i$$

$$ExFORM_{n+1} = FromFORM(n) \setminus \bigcup_{i=0}^{n} ExFORM_i$$

From Lemma 9.30 and Lemma 9.31, we have

$$\begin{aligned}\bigcup_{i=0}^{n+1} ExFORM_i &= \bigcup_{i=0}^{n} ExFORM_i \cup ExFORM_{n+1} \\ &= \bigcup_{i=0}^{n} ExFORM_i \cup (FromFORM(n) \setminus \bigcup_{i=0}^{n} ExFORM_i) \\ &= \bigcup_{i=0}^{n} ExFORM_i \cup FromFORM(n) \\ &= FORM_{n+1}\end{aligned}$$

□

Lemma 9.32.

$$\bigcup_{i\geq 0} ExFORM_i = \bigcup_{i\geq 0} FORM_i$$

Proof. $\bigcup_{i\geq 0} ExFORM_i \subseteq \bigcup_{i\geq 0} FORM_i$

$\forall n \geq 0$, From Lemma 2.28

$$\phi \in \bigcup_{i=0}^{n} ExFORM_i \to \phi \in FORM_n \to \phi \in \bigcup_{i=0}^{n} FORM_i$$

$\bigcup_{i\geq 0} ExFORM_i \supseteq \bigcup_{i\geq 0} FORM_i$

Corollary from Lemma 2.28 □

Corollary 9.33.

$$\bigcup_{i\geq 0} ExFORM_i = INDFORM$$

$$\forall \phi \in ExFORM_i \to \phi \text{ is well-formed formula}$$

10.1, 2

Claim:

$$FORMSeq = \{w \in \Sigma^* \mid w \text{ is formation-sequence-well-formed}\} = INDFORM$$

Proof. Let ρ_i denotes the set of ρ that contains a sequence of i strings $w_1, w_2, \ldots, w_i$
Define

$$str : \rho \mapsto w, str(\rho = w_1, w_2, \ldots, w_k) = w_k$$
$$str : \rho_i \mapsto \{w\}, str(\rho = w_1, w_2, \ldots, w_k) = \{w \mid w = str(\rho), \rho \in \rho_i\}$$

Claim:

$$\bigcup_{k\geq 1} str(\rho_k) = \bigcup_{i\geq 0} ExFORM_i$$

Proof. $\bigcup_{k\geq 1} \rho_k \subseteq \bigcup_{i\geq 0} ExFORM_i$

$\forall w \in FORMSeq, \exists \rho = w_1, w_2, \ldots, w_k, w = w_k$
Base case:

$$\bigcup_{k=1}^{1} str(\rho_1) = \{w \mid w \in P\} = \bigcup_{i=0}^{0} ExFORM_i$$

Inductive hypothesis: Suppose

$$\bigcup_{j=1}^{k+1} str(\rho_j) \subseteq \bigcup_{i=0}^{k} ExFORM_i$$

$$\bigcup_{j=1}^{k+2} str(\rho_j) = \bigcup_{j=1}^{k+1} str(\rho_j) \cup \{w \mid \exists \rho \in \rho_{k+2}, w = str(\rho)\}$$

By definition, $w_{k+2} = str(\rho)$ for some $\rho \in \rho_{k+2}$, one of the 3 below holds:

1. $w_{k+2} \in P \rightarrow w_{k+2} \in \bigcup_{i\geq 0} ExFORM_i$
2. $w_{k+2} = (\neg w_n), n < k+2 \rightarrow w_n \in \bigcup_{j=1}^{n} str(\rho_j) \subseteq \bigcup_{i=0}^{k} ExFORM_i$
3. $w_{k+2} = (w_m \vee w_n), m, n < k+2 \rightarrow w_m, w_n \in \bigcup_{j=1}^{\max(m,n)} str(\rho_j) \subseteq \bigcup_{i=0}^{k} ExFORM_i$

$\bigcup_{k\geq 1} \rho_k \supseteq \bigcup_{i\geq 0} ExFORM_i$

Base case is the same as above.
Inductive hypothesis: (this will be used in Q11)

Lemma 9.34.

$$\bigcup_{i=0}^{k} ExFORM_i \subseteq \bigcup_{j=1}^{2(k+1)} str(\rho_j)$$

$\forall w \in ExFORM_{k+1}$, one of the 3 below holds: 1.

$$w \in P \rightarrow w \in ExFORM_0 \rightarrow w \in \bigcup_{i=0}^{k} ExFORM_i \subseteq \bigcup_{j=1}^{k+1} str(\rho_j)$$

2.

$$\begin{aligned} w = (\neg w_n), w_n \in ExFORM_k &\rightarrow \exists \rho = w_1, w_2, \ldots, w_k \in \bigcup_{j=1}^{k+1} \rho_j, w_n = str(\rho) \\ &\rightarrow \exists \rho' \in \rho_{k+2}, \rho' = w_1, w_2, \ldots, w_k, (\neg w_n) \\ &\rightarrow w \in str(\rho_{k+2}) \rightarrow w \in \bigcup_{j=1}^{k+2} str(\rho_j) \\ &\rightarrow w \in \bigcup_{j=1}^{2(k+1)} str(\rho_j) \end{aligned}$$

3. (Concat finite sequences that yield formula composing disjunctions gives finite sequence)

$$\begin{aligned} w = (w_m \vee w_n), WLOG, w_m \in ExFORM_k, \exists q \leq k, w_n \in ExFORM_q \\ \rightarrow \exists \rho^m \in \rho_{k+1} = w_1^m, w_2^m, \ldots, w_k^m, \\ \rho^n \in \rho_{q+1} = w_1^n, w_2^n, \ldots, w_q^n \in \bigcup_{j=1}^{k+1} \rho_j, w_m = str(\rho^m), w_n = str(\rho^n) \\ \rightarrow \exists \rho' = w_1^m, w_2^m, \ldots, w_k^m, w_1^n, w_2^n, \ldots, w_q^n, (w_k^m \vee w_q^n) \in \rho_{k+q+2} \\ \rightarrow w \in \bigcup_{j=1}^{k+q+1} str(\rho_j) \\ \rightarrow w \in \bigcup_{j=1}^{2(k+1)} str(\rho_j) \end{aligned}$$

Therefore,

$$\bigcup_{k\geq 1} str(\rho_k) = \bigcup_{i\geq 0} ExFORM_i$$

□

By Corollary 9.33,

$$\bigcup_{k\geq 1} str(\rho_k) = INDFORM$$

□

Q11

Lemma 9.35. *Every well-formed formula ϕ is of one of the following forms:*

1. $\phi \in P$,
2. $\phi = (\neg\psi)$,
3. $\phi = (\psi_1 \vee \psi_2)$.

Proof. From Corollary 9.33,

$$\forall \phi \in INDFORM, \exists k \geq 0, \phi \in ExFORM_k$$

Case $x = 0$:

$$x = 0 \rightarrow \phi \in ExFORM_0 = P$$

Case $x > 0$:

$$\begin{aligned} &\phi \in ExFORM_x \text{ and } \forall y \in [0, x), \phi \notin ExFORM_y \\ &\forall \phi \in ExFORM_x, \phi \in P \text{ or} \\ &\phi \in (\{(\neg\psi) \mid \psi \in ExFORM_{x-1}\} \\ &\quad \cup \{(\psi_1 \vee \psi_2) \mid \psi_1 \in ExFORM_{x-1}, \exists j \leq x-1, \psi_2 \in ExFORM_j\}) \end{aligned}$$

Since we define $ExFORM_i$ to be pair wise disjoint, we guarantee that

$$\phi \in ExFORM_k \iff k \text{ is the minimum depth needed to form } \phi \text{ in the syntax tree}$$

Intuitively, x is the minimum depth required to formulate ϕ, thus with depth $d \in [0, x), \phi$ cannot be formed
Suppose $\phi \in P$,

$$\phi \in ExFORM_0, \phi \in ExFORM_x, x > 0$$

Contradiction. Therefore

$$\phi \in (\{(\neg\psi) \mid \psi \in ExFORM_{x-1}\} \cup \{(\psi_1 \vee \psi_2) \mid \psi_1 \in ExFORM_{x-1}, \exists j \leq x-1, \psi_2 \in ExFORM_j\})$$

Now we will prove $\forall \phi \in INDFORM$

$$(\phi \in \{(\neg\psi) \mid \psi \in ExFORM_{x-1}\}) \oplus (\phi \in \{(\psi_1 \vee \psi_2) \mid \psi_1 \in ExFORM_{x-1}, \exists j \leq x-1, \psi_2 \in ExFORM_j\})$$

Suppose we enforce the use of parentheses for all subformula, that is except $\phi \in P$, all formula starts with $'('$. If for some i, $\phi \in \{(\neg\psi) \mid \psi \in ExFORM_i\}$, $'\neg'$ must be at index 1. Likewise, if for some $i, \phi \in \{(\psi_1 \vee \psi_2) \mid \psi_1 \in ExFORM_i, \exists j \leq i, \psi_2 \in ExFORM_j\}$, $'\neg'$ must not be at index 1. Thus $\nexists \phi \in INDFORM, \phi \in \{(\neg\psi) \mid \psi \in ExFORM_{x-1}\}$ and $\phi \in \{(\psi_1 \vee \psi_2) \mid \psi_1 \in ExFORM_i, \exists j \leq i, \psi_2 \in ExFORM_j\}$ □

HW2

Lemma 9.36 (Characterization of Well-Formed Formulae)**.** *Let P be a set of propositions and $C = \{\neg, \vee\}$. Every well-formed formula ϕ over (P, C) is of exactly one of the following forms:*

1. *$\phi \in P$,*
2. *$\phi = (\neg\psi)$ for some well-formed formula ψ,*
3. *$\phi = (\psi_1 \vee \psi_2)$ for some well-formed formulae ψ_1, ψ_2.*

Structural Induction Principle. A property $P(\phi)$ holds for all well-formed formulae if:

- $P(p)$ holds for all $p \in P$,
- $P(\psi)$ implies $P(\neg\psi)$,
- $P(\psi_1)$ and $P(\psi_2)$ imply $P(\psi_1 \vee \psi_2)$.

Q1

1.1

Theorem 9.37. *Let α be a well-formed formula. Let $c(\alpha)$ denote the number of occurrences of binary connective symbols $(\wedge, \vee, \rightarrow, \leftrightarrow)$ in α, and let $s(\alpha)$ denote the number of occurrences of sentence symbols in α. Then*

$$s(\alpha) = c(\alpha) + 1.$$

Proof. **Base Case:**

Let $\alpha = p$ for some proposition $p \in \mathcal{P}$.

- $s(p) = 1$ and $c(p) = 0$.
- Since $1 = 0 + 1$, $P(p)$ holds for all atomic propositions.

Inductive Case 1 (Negation):

Suppose $P(\psi)$ holds, i.e., $s(\psi) = c(\psi) + 1$. Let $\alpha = (\neg\psi)$.

- $s(\neg\psi) = s(\psi)$ no new sentence symbols are added.
- $c(\neg\psi) = c(\psi)$ $\neg$ not binary connective.
- Thus, $s(\alpha) = s(\psi) = c(\psi) + 1 = c(\alpha) + 1$. $P(\alpha)$ holds.

Inductive Case 2 (Binary Connectives):

Suppose $P(\psi_1)$ and $P(\psi_2)$ hold. Let $\alpha = (\psi_1 c \psi_2)$ where $c \in \{\vee, \rightarrow\}$.

- $s(\alpha) = s(\psi_1) + s(\psi_2)$ (sum of occurrences in sub-formulae).
- $c(\alpha) = c(\psi_1) + c(\psi_2) + 1$ (sum of binary connectives plus the new $\star$).
- By the inductive hypothesis: $s(\alpha) = (c(\psi_1) + 1) + (c(\psi_2) + 1)$.
- Rearranging gives $s(\alpha) = (c(\psi_1) + c(\psi_2) + 1) + 1$.
- Therefore, $s(\alpha) = c(\alpha) + 1$. $P(\alpha)$ holds.

□

1.2

We define the property $P(\alpha)$ as follows: the length of α, denoted $L(\alpha)$, is of the form $4k+1$ and the number of sentence symbols, $s(\alpha)$, is $k + 1$ for some $k \in \mathbb{N}$

Proof. **Base Case:**

Let $\alpha = p$ for some proposition $p \in \mathcal{P}$

- The length $L(p) = 1 = 4 \cdot 0 + 1$
- The number of sentence symbols $s(p) = 1$
- Thus, $P(p)$ holds.

Inductive Case:

Suppose $P(\psi_1)$ and $P(\psi_2)$ hold Then $L(\psi_1) = 4k_1 + 1$ and $s(\psi_1) = k_1 + 1$. Also $L(\psi_2) = 4k_2 + 1$ and $s(\psi_2) = k_2 + 1$. Let $\alpha = (\psi_1 c \psi_2)$ where c is a binary connective

Length of α:

- The string α consists of: one left parenthesis, ψ_1, one binary connective, ψ_2, and one right parenthesis
- $L(\alpha) = 1 + L(\psi_1) + 1 + L(\psi_2) + 1$.
- $L(\alpha) = 3 + (4k_1 + 1) + (4k_2 + 1) = 4k_1 + 4k_2 + 5$.
- $L(\alpha) = 4(k_1 + k_2 + 1) + 1$.
- Let $k = k_1 + k_2 + 1$, we see $L(\alpha) = 4k + 1$, odd

Number of sentence symbols:

- $s(\alpha) = s(\psi_1) + s(\psi_2) = (k_1 + 1) + (k_2 + 1) = k_1 + k_2 + 2$.
- Using $k = k_1 + k_2 + 1$ from above, $s(\alpha) = (k_1 + k_2 + 1) + 1 = k + 1$
- $k + 1 = \frac{4k+4}{4} > \frac{4k+1}{4}$

□

Q2

Recall the following derived connectives:

- $\phi_1 \wedge \phi_2 \equiv \neg\big((\neg\phi_1) \vee (\neg\phi_2)\big)$,
- $\bot \equiv \neg(p \vee \neg p)$ for some $p \in P$,
- $\phi_1 \to \phi_2 \equiv (\neg\phi_1) \vee \phi_2$.

Using structural induction, show that for every formula ϕ over $(P, \{\neg, \vee\})$ with $P \neq \emptyset$, there exists a formula ψ using only connectives $\{\bot, \to\}$ such that ϕ and ψ are logically equivalent.

Proof. We will show that every formula formed over $\{\neg, \vee\}$ can be formed using $\{\bot, \to\}$, and denote the logically equivalent formula using the later connector set from ϕ as ψ Define property $P(\phi)$ as follows: there exists a logically equivalant formula over P using the connector set $\{\bot, \to\}$.

Base Case:

- $\forall \phi = p \in P, \psi = p$

Inductive Case 1:

Suppose $P(\phi)$ for some formula, $\exists \psi \equiv \phi$

- $(\neg\phi) = (\neg\phi) \vee ((\neg\phi) \vee (\neg\phi))) = \phi \to \bot = \psi \to \bot$

Inductive Case 2:

Suppose $P(\phi_1), P(\phi_2)$for some formula, $\exists \psi_1 \equiv \phi_1, \psi_2 \equiv \phi_2$

- $(\phi_1 \vee \phi_2) = ((\neg(\neg\phi_1)) \vee \phi_2) = ((\neg\phi_1) \to \phi_2)$
- From inductive case 1, $((\neg\phi_1) \to \phi_2) = ((\phi_1 \to \bot) \to \phi_2) = ((\psi_1 \to \bot) \to \psi_2)$

□

Q3

Definition 9.38 (Conjunctive Normal Form)**.** *A formula ϕ over $(P, \{\neg, \vee, \wedge\})$ is in* conjunctive normal form (CNF) *if*

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m \quad (m \geq 1),$$

where each clause C_i has the form

$$C_i = \ell_{i,1} \vee \ell_{i,2} \vee \cdots \vee \ell_{i,k_i},$$

and each literal $\ell_{i,j}$ is either p or $\neg p$ for some $p \in P$.

Using structural induction, show that for every well-formed formula ϕ, there exists a logically equivalent formula ψ in CNF.

Proof. Define property $P(\phi)$ to be there exists a CNF ψ that is logically equivalent to ϕ

Base Case:

$\phi = p$. An atomic proposition p is a clause containing a single literal. A single clause is, by definition, a CNF. Thus, $P(p)$ holds.

Inductive Case 1 $\neg$:

Assume $P(\phi)$ holds. If ϕ has a CNF, we can first convert $\neg\phi$ into NNF https://en.wikipedia.org/wiki/Negation_normal_form

- $\neg(A \wedge B) \equiv \neg A \vee \neg B$
- $\neg(A \vee B) \equiv \neg A \wedge \neg B$
- $\neg\neg A \equiv A$

Inductive Case 2 $\vee$:

Assume $P(\phi_1)$ and $P(\phi_2)$ hold. Let $\psi_1 = \bigwedge_i C_i$ and $\psi_2 = \bigwedge_j D_j$ be CNFs.

- $\phi_1 \vee \phi_2 \equiv (\bigwedge_i C_i) \vee (\bigwedge_j D_j)$.
- By the Distributive Law $(A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C))$, we distribute the $\vee$ over the $\wedge$:
- $\phi_1 \vee \phi_2 \equiv \bigwedge_i \bigwedge_j (C_i \vee D_j)$.
- Since the disjunction of two clauses $C_i \vee D_j$ is also a clause, the final expression is a CNF.

□

Q4

Definition 9.39 (Occurs)**.** *Define $OCCUR : \text{FORM} \to \mathcal{P}(P)$ inductively by:*

- *If $\phi = p \in P$, then $OCCUR(\phi) = \{p\}$,*
- *If $\phi = \neg\psi$, then $OCCUR(\phi) = OCCUR(\psi)$,*
- *If $\phi = \psi_1 \oplus \psi_2$ with $\oplus \in \{\wedge, \vee\}$, then $OCCUR(\phi) = OCCUR(\psi_1) \cup OCCUR(\psi_2)$.*

Lemma 9.40 (Relevance Lemma)**.** *Let ϕ be a propositional formula and let $v_1, v_2 : P \to \{\text{true}, \text{false}\}$ be valuations such that $v_1(p) = v_2(p)$ for all $p \in OCCUR(\phi)$. Then $v_1 \models \phi$ if and only if $v_2 \models \phi$.*

Prove this lemma using structural induction.

Proof. We define the property $P(\phi)$ as: for any v_1, v_2, if $v_1(p) = v_2(p)\ \forall p \in OCCUR(\phi)$, $v_1 \models \phi \iff v_2 \models \phi$.

Base Case:

$\phi = p$ for some $p \in P$

- $OCCUR(p) = \{p\}$, $v_1(p) = v_2(p)$
- $v_1 \models p \iff v_1(p) = \text{true}$ and $v_2 \models p \iff v_2(p) = \text{true}$
- $v_1(p) = v_2(p)$, $v_1 \models p \iff v_2 \models p$

Inductive Step 1 (negation):

Assume $P(\psi)$ holds. Let $\phi = \neg\psi$.

- $OCCUR(\phi) = OCCUR(\psi)$, $v_1 \models \psi \iff v_2 \models \psi$
- $v_1 \models \neg\psi \iff v_1 \not\models \psi$ and $v_2 \models \neg\psi \iff v_2 \not\models \psi$
- Since the truth values of ψ are the same, so do their negation.

Inductive Step 2 (Binary connective):

Assume $P(\psi_1)$ and $P(\psi_2)$ hold.

- Let $\phi = \psi_1 \oplus \psi_2$ for $\oplus \in \{\wedge, \vee\}$
- $OCCUR(\phi) = OCCUR(\psi_1) \cup OCCUR(\psi_2)$, if v_1, v_2 agree on $OCCUR(\phi)$, they agree on both $OCCUR(\psi_1)$ and $OCCUR(\psi_2)$
- $v_1 \models \psi_1 \iff v_2 \models \psi_1$ and $v_1 \models \psi_2 \iff v_2 \models \psi_2$
- the satisfaction of $\psi_1 \oplus \psi_2$ depends only on the truth values of ψ_1 and ψ_2, and those values are the same for v_1 and v_2, thus $v_1 \models \phi \iff v_2 \models \phi$

□

Q5

Lemma 9.41 (Generalized Relevance Lemma)**.** *Let ϕ be a propositional formula and let v_1, v_2 be valuations such that $v_1(p) = v_2(p)$ for all p in some set $S \supseteq OCCUR(\phi)$. Then $v_1 \models \phi$ if and only if $v_2 \models \phi$.*

Prove this lemma.

Proof. Let ϕ be a formula and S be a set such that $S \supseteq OCCUR(\phi)$. Suppose $v_1(p) = v_2(p) \forall p \in S$. Since S contains all elements of $OCCUR(\phi)$, the condition $v_1(p) = v_2(p) \forall p \in S \implies v_1(p) = v_2(p) \forall p \in OCCUR(\phi)$. By the Relevance lemma, if v_1 and v_2 agree on $\forall p \in OCCUR(\phi)$, then $v_1 \models \phi \iff v_2 \models \phi$. □

Q6

Lemma 9.42 (Relevance Lemma for Logical Equivalence)**.** *Let ϕ_1, ϕ_2 be propositional formulae. The following are equivalent:*

- *For all valuations v_1, v_2 agreeing on $OCCUR(\phi_1) \cup OCCUR(\phi_2)$, we have $v_1 \models \phi_1$ iff $v_2 \models \phi_2$.*
- *ϕ_1 and ϕ_2 are logically equivalent.*

Prove this lemma.

Proof. We prove the equivalence of the two statements by showing both implications.

($\Leftarrow$) Suppose ϕ_1 and ϕ_2 are logically equivalent. $\forall v,\ v \models \phi_1 \iff v \models \phi_2$. Consider v_1, v_2 that agree on $OCCUR(\phi_1) \cup OCCUR(\phi_2)$, we have:

- $v_1 \models \phi_1 \iff v_1 \models \phi_2$
- $v_2 \models \phi_1 \iff v_2 \models \phi_2$

Trivial

($\Rightarrow$) Suppose $\forall v_1, v_2$ agreeing on $S = OCCUR(\phi_1) \cup OCCUR(\phi_2)$, we have $v_1 \models \phi_1 \iff v_2 \models \phi_2$. Let v be any valuation. We will show that $v \models \phi_1 \iff v \models \phi_2$. Define $v_1 = v$ and $v_2 = v$. These valuations trivially agree on S.

- $v_1 \models \phi_1 \iff v_2 \models \phi_2$
- $v \models \phi_1 \iff v \models \phi_2$
- ϕ_1 and ϕ_2 are logically equivalent

□

Q7

Let $G = (V, E)$ be an undirected graph and $k \in \mathbb{N}$. Construct a propositional formula ϕ_G such that:

1. $|\phi|$ is polynomial in $k + |V| + |E|$,
2. ϕ is satisfiable if and only if G has a vertex cover of size at most k.

Formally prove correctness of the encoding.

Proof. Define the proposition set P :

$$
\begin{aligned}
&P = P_{in} \cup P_{out} \cup P_{count} \\
&P_{in} = \{p_{v,in} \mid v \in V\} \\
&P_{out} = \{p_{v,out} \mid v \in V\} \\
&P_{count} = \{c_{i,j} \mid i, j \in [0, |V|]\} \\
&|P| \in O(|V|^2)
\end{aligned}
$$

There are constraint we need to consider:

$\forall v \in V, v \in V'$ **or** $v \in V \setminus V'$

$$\phi_{in} = \bigwedge_{v \in V} (p_{v,in} \vee p_{v,out})$$

$|\phi_{in}| \in O(|V|)$

$\forall v \in V, v \notin V'$ **or** $v \notin V \setminus V'$

$$\phi_{out} = \bigwedge_{v \in V} \neg(p_{v,in} \wedge p_{v,out})$$

$|\phi_{out}| \in O(|V|)$

All edges covered

$$\phi_e = \bigwedge_{\{u,v\} \in E} (p_{u,in} \vee (p_{v,in}))$$

$|\phi_e| \in O(|V|^2)$

$|V'| \leq k$

We use the following facts to define inductive rules:

- The number of true proposition is non decreasing as i increases in the sequence $p_1, \ldots, p_i$. $\rightarrow$ if the $c_{i,j}$ is true, $\forall \ell \geq i, c_{\ell,j}$ is true.
- if the $c_{i,j}$ is false, $\forall \ell \geq j, c_{i,\ell}$ is false

Inductively define $c_{i,j}$:

$$[c_{i-1,j} \vee (c_{i-1,j-1} \wedge p_{i,in})] \leftrightarrow c_{i,j}$$

We use bidirectional implication to prevent vacuous truthity of $c_{i,j}$ when conditions do not hold. And the last constraint can be represented by $\neg(c_{|V|,k+1})$
$|\{c_{i,j} \mid i \in [1, |V|], j \in [1, k+1]\}| \in O(|V| \cdot k)$
Putting everything together we can encode k-vertex cover using

$$\phi = \phi_{in} \wedge \phi_{out} \wedge \phi_e \wedge (\neg c_{|V|,k+1})$$

□

Correctness of encoding

To prove the correctness of the encoding, we will show:

Theorem 9.43. *ϕ is satisfiable $\rightarrow$ $\exists k$-vertex cover on G*

Proof. ϕ is satisfiable $\rightarrow \exists v, v \models \phi, \rightarrow$

$$v \models \phi_{in}$$
$$v \models \phi_{out}$$
$$v \models \phi_e$$
$$v \models (\neg c_{|V|,k+1})$$

As per construction, this guarentees $\forall v \in V$, v is either in the vertex cover or exclusively; $\forall e = \{u, v\} \in E$, either u or v inclusively are in the vertex cover, and the size of the cover is at most k □

Theorem 9.44. *$\exists k$-vertex cover on $G \rightarrow \phi$ is satisfiable*

Proof. Suppose $G = (V, E)$ has a vertex cover $V' \subseteq V$ such that $|V'| \leq k$. We construct a valuation v as follows:

1. For each $u \in V$, set $v(p_{u,in}) =$ true if $u \in V'$, and $v(p_{u,in}) =$ false otherwise
2. Set $v(p_{u,out}) = \neg v(p_{u,in}) \implies \phi_{in} \wedge \phi_{out}$ is satisfiable
3. Since V' is a vertex cover, for every edge $\{u, v\} \in E$, at least one of u or v must be in V'. Thus, $v(p_{u,in}) \vee v(p_{v,in})$ is true for all edges, satisfying ϕ_e.
4. Define $v(c_{i,j})$ to be true if and only if $|\{u_1, \ldots, u_i\} \cap V'| \geq j$.
    - the recurrence $[c_{i-1,j} \vee (c_{i-1,j-1} \wedge p_{i,in})] \leftrightarrow c_{i,j}$ is satisfied because the count of vertices in the cover up to i increases only if the i-th vertex is in the cover.
    - Since $|V'| \leq k$, the total count of vertices in V' after considering all $|V|$ vertices cannot be $k+1$. Thus, $v(c_{|V|,k+1}) =$ false, which satisfies $\neg c_{|V|,k+1}$.

Since all components are satisfied, ϕ is satisfiable. □

Q8

Given a partial $N^2 \times N^2$ Sudoku grid S, construct a propositional formula ϕ_S of size polynomial in N^2 such that ϕ_S is satisfiable if and only if S has a completion to a valid Sudoku grid. Prove correctness.

Proof. Define proposition set $P = \{p_{i,j,k} \mid i, j, k \in [1, N]\}$, representing cell (i, j) is assigned the number k We need to satisfiy the following constriants:

Cell: At most 1 value assigned

$$\phi_{cell,\leq 1} = \bigwedge_{i=1}^{N} \bigwedge_{j=1}^{N} \bigwedge_{1 \leq k \leq m \leq N} (\neg p_{i,j,k} \vee \neg p_{i,j,m})$$

Cell: At least 1 value assigned

$$\phi_{cell,\geq 1} = \bigwedge_{i=1}^{N} \bigwedge_{j=1}^{N} \bigvee_{k=1}^{N} p_{i,j,k}$$

Row: At most 1 value assigned

$$\phi_{row,\leq 1} = \bigwedge_{i=1}^{N} \bigwedge_{k=1}^{N} \bigwedge_{1 \leq j < m \leq N} (\neg p_{i,j,k} \vee \neg p_{i,m,k})$$

Column: At most 1 value assigned

$$\phi_{col,\leq 1} = \bigwedge_{j=1}^{N} \bigwedge_{k=1}^{N} \bigwedge_{1 \leq i < m \leq N} (\neg p_{i,j,k} \vee \neg p_{m,j,k})$$

Block: At most 1 value assigned

Let $n = \sqrt{N}$. For each block starting at row $r \in \{0, \ldots, n-1\}$ and column $c \in \{0, \ldots, n-1\}$:

$$\phi_{block,\leq 1} = \bigwedge_{r=0}^{n-1} \bigwedge_{c=0}^{n-1} \bigwedge_{k=1}^{N} \bigwedge_{(i,j) \neq (i',j')} (\neg p_{rn+i,cn+j,k} \vee \neg p_{rn+i',cn+j',k})$$

where $1 \leq i, j, i', j' \leq n$.

Initial Setup

Let S be the set of triples (i, j, k) such that cell (i, j) is pre-assigned value k:

$$\phi_{clues} = \bigwedge_{(i,j,k) \in S} p_{i,j,k}$$

The complete formula is:

$$\phi_{Sudoku} = \phi_{cell,\leq 1} \wedge \phi_{cell,\geq 1} \wedge \phi_{row,\leq 1} \wedge \phi_{col,\leq 1} \wedge \phi_{block,\leq 1} \wedge \phi_{clues}$$

□

Correctness of Encoding

Theorem 9.45. *The formula ϕ_{Sudoku} is satisfiable $\iff$ the Sudoku puzzle has a valid solution*

Proof. ($\Rightarrow$) Suppose ϕ_{Sudoku} is satisfiable under valuation v. We construct a Sudoku assignment by setting cell $(i, j) = k$ if $v(p_{i,j,k}) = \text{true}$.

- By $\phi_{cell,\geq 1}$ and $\phi_{cell,\leq 1}$, every cell is assigned exactly one value.
- By $\phi_{row,\leq 1}$, no value k is repeated in a row. Since there are N cells in a row and N possible values, each value must appear exactly once.

- The same logic applies to $\phi_{col,\leq 1}$ and $\phi_{block,\leq 1}$ for columns and $n \times n$ subgrids.
- ϕ_{clues} ensures the solution respects the starting digits.

($\Leftarrow$) Suppose a valid Sudoku solution exists. Define a valuation v such that $v(p_{i,j,k}) =$ true if cell (i, j) contains k, and false otherwise.

- Because the solution is valid, each cell has one number (satisfying cell constraints).
- Because rows, columns, and blocks contain no duplicates, the "at most one" (≤ 1) constraints for each are satisfied.
- Because the solution contains the initial clues, ϕ_{clues} is satisfied.

Therefore, $v \models \phi_{Sudoku}$. □

Background: Compactness

Definition 9.46 (Satisfiable)**.** *A set Γ of propositional formulae is satisfiable if there exists a valuation v such that $v \models \Gamma$.*

Definition 9.47 (Finitely Satisfiable)**.** *A set Γ is finitely satisfiable if every finite $\Gamma' \subseteq \Gamma$ is satisfiable.*

Theorem 9.48 (Compactness Theorem)**.** *A set Γ of propositional formulae is satisfiable if and only if it is finitely satisfiable.*

Q11

Lemma 9.49. *If Γ is satisfiable, then Γ is finitely satisfiable.*

Proof. Γ is satisfiable $\implies \exists v, \forall \phi \in \Gamma, v \models \phi$
$\forall$ finite subset $\Gamma' \subseteq \Gamma, \forall \psi \in \Gamma', \psi \in \Gamma \implies \forall \Gamma' \subseteq \Gamma, \forall \psi \in \Gamma;, v \models \psi$ □

Q12

Definition 9.50 (P-complete)**.** *Γ is P-complete if for every $p \in P$, either $p \in \Gamma$ or $\neg p \in \Gamma$.*

Definition 9.51 (P-consistent)**.** *Γ is P-consistent if for no $p \in P$ do both p and $\neg p$ belong to Γ.*

Lemma 9.52. *If Γ is finitely satisfiable and P-complete, then Γ is P-consistent.*

1. Prove the lemma.

Proof. Suppose otherwise, Γ is P-complete, finitely satisfiable and NOT P-consistent, $\exists$finite set $\Gamma' \subseteq \Gamma, \exists p \in P, p \in \Gamma', (\neg p) \in \Gamma' \implies \Gamma'$ is not satisfiable, a contradiction. □

12.2

- $P = \{p, \}$
- $\Gamma = \emptyset$

12.3

- $P = \{p\}$
- $\Gamma = \{(\neg(p \vee (\neg p)))\}$

12.4

I cannot

Q13

Let Γ be finitely satisfiable and let ϕ be a formula.

1. Prove that at least one of $\Gamma \cup \{\phi\}$ or $\Gamma \cup \{\neg\phi\}$ is finitely satisfiable.
2. Give examples as requested.

Lemma 9.53. *at least one of $\Gamma \cup \{\phi\}$ or $\Gamma \cup \{\neg\phi\}$ is finitely satisfiable.*

Proof. We will prove the contrapositive of this statement: if neither $\Gamma \cup \{\phi\}$ nor $\Gamma \cup \{(\neg\phi)\}$ is satisfiable, Γ is not finitely satisfiable. Consider finite subset $\Gamma_1, \Gamma_2 \subseteq \Gamma$, and $\phi \in FORM, \Gamma_1 \cup \{\phi\}$ and $\Gamma_2 \cup \{(\neg\phi)\}$ are both not satisfiable, $\forall v, v \not\models (\Gamma_1 \cup \{\phi\}) \implies v \not\models \Gamma_1$ or $v \not\models \phi$, and $\forall v, v \not\models (\Gamma_2 \cup \{(\neg\phi)\}) \implies v \not\models \Gamma_2$ or $v \not\models (\neg\phi)$
Rewrite with respect to the set of all valuations, define

- $V_1 = \{v : P \mapsto \{T, F\} \mid v \models \Gamma_1\}$
- $V_2 = \{v : P \mapsto \{T, F\} \mid v \models \Gamma_2\}$
- $V_\phi = \{v : P \mapsto \{T, F\} \mid v \models \{\phi\}\}$
- $V_{(\neg\phi)} = \{v : P \mapsto \{T, F\} \mid v \not\models \{\phi\}\}$

$$V_1 \cap V_\phi = \emptyset \implies \forall v \in V_1, v \not\models \phi \implies v \models (\neg\phi) \implies V_1 \subseteq V_{(\neg\phi)}$$
$$V_2 \cap V_{(\neg\phi)} = \emptyset \implies \forall v \in V_2, v \not\models (\neg\phi) \implies v \models \phi \implies V_2 \subseteq V_\phi$$

Now consider also a finite subset to $\Gamma, \Gamma_1 \cup \Gamma_2$, define

- $V_\cup = \{v : P \mapsto \{T, F\} \mid v \models (\Gamma_1 \cup \Gamma_2)\}$

$V_\cup = (V_1 \cap V_2) \subseteq (V_\phi \cup V_{(\neg\phi)}) = \emptyset \implies \nexists v, v \models (\Gamma_1 \cup \Gamma_2) \implies \Gamma$ is not finitely satisfiable. □

13.2

- $\Gamma = \{p\}, \phi = q$

13.3

- $\Gamma = \{p, (\neg q)\}, \phi = q$

Q14, 15

Let $P = \{p_1, p_2, \dots\}$ and define $\Gamma_0, \Gamma_1, \dots$

Lemma 9.54. $\Gamma_U = \bigcup_{i \geq 0} \Gamma_i$ *is finitely satisfiable.*

Lemma 9.55.

$$\forall i \geq 0, \Gamma_i \text{ is fintiely satisfiable}$$

Proof. Base case: $i = 0, \Gamma_i = \Gamma_0 = \Gamma$ is finitely satisfiable by definition.
Inductive Hypothesis: Γ_n is finitely satisfiable. By Lemma 9.53, $\Gamma_n \cup \{p_n\}$ is finitely satisfiable or $\Gamma_n \cup \{(\neg p_n)\}$ is finitely satisfiable. □

Proof. Consider finite set $\Gamma' \subseteq \Gamma_U$, since Γ' is finite, and $\forall k \geq 0, \Gamma_k \subseteq \Gamma_{k+1}$ by definition, $\exists j \geq 0, \Gamma' \subseteq \Gamma_j = \bigcup_{i \geq 0}^{j} \Gamma_i$. By Lemma 9.49, Γ_j is satisfiable $\to \Gamma'$ is finitely satisfiable. □

Proof. From the construction, $\forall i \geq 0$ at least one of p_i and $(\neg p_i)$ is in $\bigcup_{i\geq 0}\Gamma_i$. Since $\bigcup_{i\geq 0}\Gamma_i$ is finitely satisfiable, $\forall i \geq 0, p_i$ and $(\neg p_i)$ cannot both be in $\bigcup_{i\geq 0}\Gamma_i$. Otherwise $\{p_i\} \cup \{(\neg p_1)\} \subseteq \bigcup_{i\geq 0}\Gamma_i$ is not satisfiable, contradiction. □

Now we have proven that the construction proposed from a finitely satisfiable formula set Γ to $\bigcup_{i\geq 0}\Gamma_i$ is also finitely satisfiable. Now to complete the proof of the compactness theorem, we need to prove that $\exists v, v \models \Gamma$.

Proof. From Lemma 9.54, $\forall i \geq 0, (p_i \in \Gamma_U) \oplus ((\neg p_i) \in \Gamma_U)$, $\exists v, v \models \Gamma_U$

$$v(p) : P \mapsto \{T, F\} = \begin{cases} T, & \text{if } p \in \Gamma_U \\ F, & \text{if } (\neg p) \in \Gamma_U \end{cases}$$

Define the literal set $Literals = \{p_i, (\neg p_i) \mid i \geq 0\}$, and function

$$\tau(p) : P \mapsto Literals = \begin{cases} p, & \text{if } p \in \Gamma_U \\ (\neg p), & \text{if } (\neg p) \in \Gamma_U \end{cases}$$

Lemma 9.56.

$$v \models \Gamma_U \rightarrow v \models \tau(\Gamma)$$

Let $\phi \in \Gamma$, Define literal set $X = \{\tau(p) \mid p \in OCCUR(\phi)\}$ where X is a set containing single proposition literals. By definition we have

$$X \subseteq \Gamma_U \rightarrow v \models X \rightarrow \forall \psi \in X, v(\psi) = T$$

and $OCCUR(X) = OCCUR(\phi)$ Since both X is finite, $X \cup \{\phi\}$ is a finite subset to Γ_U, thus satisfiable. $\rightarrow \exists v', v' \models X \cup \{\phi\} \rightarrow v' \models X, v' \models \phi \rightarrow \forall \psi \in X, v'(\psi) = T$.

$$\forall p \in OCCUR(\phi), v(p) = v'(p), v' \models \phi \rightarrow v \models \phi$$

□

Q16

Let $\top$ and $\bot$ be the 0-place connectives (truth and falsity). For a wff ϕ and sentence symbol A, let $\phi^A_\top$ and $\phi^A_\bot$ denote the substitution of A with $\top$ and $\bot$ respectively. Define the formula $\phi^*_A = (\phi^A_\top \vee \phi^A_\bot)$.

Proof. **1. $\phi \models \phi^*_A$.**

Let v be a valuation such that $v \models \phi$.

- If $v(A) = T \implies v(\phi) = v(\phi^A_\top)$. $v \models \phi \implies v \models \phi^A_\top$.
- If $v(A) = F \implies v(\phi) = v(\phi^A_\bot)$. $v \models \phi \implies v \models \phi^A_\bot$.
- either way, $v \models (\phi^A_\top \vee \phi^A_\bot)$ Thus, $\phi \models \phi^*_A$.

2. If $\phi \models \psi$ and A does not occur in ψ, then $\phi^*_A \models \psi$.

Let $v \models \phi^*_A \implies v \models \phi^A_\top$ or $v \models \phi^A_\bot$.

- Case 1: $v \models \phi^A_\top$. Let v' be valuation $v[A \mapsto \text{true}] \implies v' \models \phi$. $\phi \models \psi \implies v' \models \psi$. $A \notin OCCUR(\psi) \implies \psi$ is independent of $A \implies v \models \psi$.
- Case 2: similar
- Thus, $\phi^*_A \models \psi$

3. ϕ is satisfiable iff ϕ^*_A is satisfiable.

- ($\Rightarrow$) ϕ is satisfiable $\implies \exists v \models \phi$. By part 1, $\phi \models \phi^*_A \implies v \models \phi^*_A$
- ($\Leftarrow$) ϕ^*_A is satisfiable $\implies \exists v, v \models \phi^A_\top$ or $v \models \phi^A_\bot$. $v \models \phi^A_\top \implies v[A \mapsto \text{true}] \models \phi$. $v \models \phi^A_\bot \implies v[A \mapsto \text{false}] \models \phi$

□

Theorem 9.57 (Interpolation Theorem)**.** *If $\alpha \models \beta$, then there exists a formula γ such that $\alpha \models \gamma \models \beta$, and $OCCUR(\gamma) \subseteq OCCUR(\alpha) \cap OCCUR(\beta)$.*

Proof. Let $S = OCCUR(\alpha) \setminus OCCUR(\beta)$,

- Let $\alpha_0 = \alpha$.
- $\forall i \in [1, n]$, let $\alpha_i = (\alpha_{i-1})^*_{A_i}$.
- By part 1, $\alpha_{i-1} \models \alpha_i$, by transitivity $\alpha \models \alpha_n$.
- By part 2, $\alpha \models \beta$ and $A_1 \notin OCCUR(\beta) \implies \alpha_1 \models \beta$. Repeat $\forall A_i \in S, \alpha_n \models \beta$.
- $\gamma = \alpha_n$ contains no propositions from S, meaning $OCCUR(\gamma) \subseteq OCCUR(\alpha) \cap OCCUR(\beta)$.

□

HW3

Hilbert-style Proof System

The Hilbert-style (or Frege) proof system for propositional logic comprises of 3 axiom schema and the modus ponens rule of inference

- **Axiom Schema 1:** $\varphi \to (\psi \to \varphi)$
- **Axiom Schema 2:** $(\varphi \to (\psi \to \rho)) \to ((\varphi \to \psi) \to (\varphi \to \rho))$
- **Axiom Schema 3:** $((\varphi \to \bot) \to \bot) \to \varphi$
- **Modus Ponens:** From φ and $\varphi \to \psi$, infer ψ

Question 1

Let Γ, Γ' be sets of propositional logic formulae and let φ be a formula. Show that if $\Gamma \vdash \varphi$ and if $\Gamma \subseteq \Gamma'$, then $\Gamma' \vdash \varphi$

Proof. $\Gamma \vdash \varphi \implies \exists$ a proof for $\phi, \langle \phi_1, \phi_2, \ldots, \phi_n \rangle, \phi_n = \varphi$

Base case: $n = 1$

- Case 1: $\varphi \in \Gamma \implies \varphi \in \Gamma' \implies \Gamma' \vdash \varphi$
- Case 2: φ is from Axiom 1 or 2 or 3, $\vdash \varphi \implies \Gamma' \vdash \varphi$

Inductive case: Suppose $\forall i < n, \Gamma \vdash \phi_i \implies \Gamma' \vdash \phi_i$

- Case 1: φ is from axiom 1, 2, 3, trivial
- Case 2: φ is from Modus Ponens, $\exists i, j < n, \phi_i \equiv \phi_j \to \varphi, \Gamma \vdash \phi_i, \Gamma \vdash \phi_j$

$$\frac{\phi_j \to \varphi \qquad \phi_j}{\varphi}$$

From indeuctive hypothesis, $\Gamma' \vdash \phi_i, \Gamma' \vdash \phi_j, \Gamma' \vdash \varphi$

□

Question 2

Theorem 9.58 (Deduction Theorem)**.** *Let Γ be a set of propositional logic formulae and φ, ψ be propositional logic formulae.* $\Gamma \vdash (\varphi \to \psi)$ *iff* $\Gamma \cup \{\varphi\} \vdash \psi$.

Proof. ($\implies$)

1. $\Gamma \vdash \varphi \to \psi$ (Given)
2. $\Gamma \cup \{\varphi\} \vdash \varphi \to \psi$ (Monotonicity)
3. $\Gamma \cup \{\varphi\} \vdash \varphi$ (Assumption)
4. $\Gamma \cup \{\varphi\} \vdash \psi$ (MP 2, 3)

($\impliedby$)

Let $\pi = \psi_1, \ldots, \psi_n$ be a proof of ψ from $\Gamma \cup \{\varphi\}$. Assume $\forall j < i, \Gamma \vdash \varphi \to \psi_j$.

Case 1: $\psi_i \in \Gamma$ **or** ψ_i **is an Axiom**

1. $\Gamma \vdash \psi_i$ (Assumption/Axiom)
2. $\Gamma \vdash \psi_i \to (\varphi \to \psi_i)$ (Ax 1)
3. $\Gamma \vdash \varphi \to \psi_i$ (MP 1, 2)

Case 2: $\psi_i = \varphi$

1. $\Gamma \vdash (\varphi \to ((\varphi \to \varphi) \to \varphi)) \to ((\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi))$ (Ax 2)
2. $\Gamma \vdash \varphi \to ((\varphi \to \varphi) \to \varphi)$ (Ax 1)
3. $\Gamma \vdash (\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi)$ (MP 1, 2)
4. $\Gamma \vdash \varphi \to (\varphi \to \varphi)$ (Ax 1)
5. $\Gamma \vdash (\varphi \to \varphi)$ (MP 3, 4)

Case 3: ψ_i **via MP from** ψ_j, ψ_k **where** $\psi_j = \psi_k \to \psi_i$

1. $\Gamma \vdash \varphi \to (\psi_k \to \psi_i)$ (IH)
2. $\Gamma \vdash (\varphi \to (\psi_k \to \psi_i)) \to ((\varphi \to \psi_k) \to (\varphi \to \psi_i))$ (Ax 2)
3. $\Gamma \vdash (\varphi \to \psi_k) \to (\varphi \to \psi_i)$ (MP 1, 2)
4. $\Gamma \vdash \varphi \to \psi_k$ (IH)
5. $\Gamma \vdash \varphi \to \psi_i$ (MP 3, 4)

□

Question 3

(a) $\vdash \varphi \to \varphi$

Done in Q2

(b) $\vdash (\psi \to \varphi) \to \varphi$, where $\psi = \varphi \to \bot$

Proof. $\equiv \{\psi \to \varphi\} \vdash \varphi$.

1. $(\varphi \to \bot) \to \varphi$ (Assumption)
2. $((\varphi \to \bot) \to \bot) \to \varphi$ (Axiom 3)
3. $((\varphi \to \bot) \to \varphi) \to (((\varphi \to \bot) \to (\varphi \to \bot)) \to ((\varphi \to \bot) \to \bot))$ (Axiom 2)
4. $((\varphi \to \bot) \to (\varphi \to \bot)) \to ((\varphi \to \bot) \to \bot)$ (MP 1, 3)
5. $(\varphi \to \bot) \to (\varphi \to \bot)$ (Part a)
6. $(\varphi \to \bot) \to \bot$ (MP 4, 5)
7. φ (MP 2, 6)

(Wikipedia: Peirce's law). □

(c) $\{\bot\} \vdash \varphi$

Proof.

1. $\bot$ (Assumption)
2. $\bot \to ((\varphi \to \bot) \to \bot)$ (Axiom 1)
3. $(\varphi \to \bot) \to \bot$ (MP 1, 2)
4. $((\varphi \to \bot) \to \bot) \to \varphi$ (Axiom 3)
5. φ (MP 3, 4)

□

(d) $\vdash \psi \to (\varphi \to \rho)$ **where** $\psi = \varphi \to \bot$

Proof. $\equiv \{\varphi \to \bot, \varphi\} \vdash \rho$.

1. $\varphi \to \bot$ (Assumption)
2. φ (Assumption)
3. $\bot$ (MP 1, 2)
4. $\bot \to ((\rho \to \bot) \to \bot)$ (Axiom 1)
5. $(\rho \to \bot) \to \bot$ (MP 3, 4)
6. $((\rho \to \bot) \to \bot) \to \rho$ (Axiom 3)
7. ρ (MP 5, 6)

□

Theorem 2 (Soundness of Hilbert-style Proof System)

Let Γ be a set of propositional logic formulae and φ be a propositional logic formula. If $\Gamma \vdash \varphi$, then $\Gamma \models \varphi$

Question 4

Prove Theorem 2. *Hint: Given a proof, inductively establish that every formula in the proof is a logical consequence of the assumptions.*

Theorem 3 (Completeness of Hilbert-style Proof System)

Let Γ be a set of propositional logic formulae and φ be a propositional logic formula. If $\Gamma \models \varphi$, then $\Gamma \vdash \varphi$

Theorem 4 (Weak Completeness of Hilbert-style Proof System)

Let φ be a propositional logic formula. If $\models \varphi$, then $\vdash \varphi$.

Definition 9.59 (Inconsistent set)**.** Γ *is inconsistent* $\iff$

$$\exists \varphi, \Gamma \vdash \varphi, \Gamma \vdash (\varphi \to \bot)$$

Question 5

Prove that two notions of a set being inconsistent are identical. That is, prove that the following two statements are equivalent

1. There is a formula φ such that $\Gamma \vdash \varphi$ and $\Gamma \vdash (\varphi \to \bot)$
2. For every formula ρ, $\Gamma \vdash \rho$

Lemma 1

Let Γ be a set of formulae and φ be a formula. If $\Gamma \cup \{\varphi \to \bot\}$ is inconsistent, then $\Gamma \vdash \varphi$

Lemma 2

Let Γ be a set of formulae and φ be a formula. If $\Gamma \cup \{\varphi\}$ is inconsistent, then $\Gamma \vdash (\varphi \to \bot)$

Lemma 3

Let Γ be a set of formulae. If Γ is inconsistent, then there is a finite subset $\Gamma' \subseteq_{fin} \Gamma$ of Γ such that Γ' is also inconsistent

Question 6

- (a) Prove Lemma 1

  Proof. From deduction thm, $\Gamma \vdash (\varphi \to \bot) \to \psi$ and $\Gamma \vdash (\varphi \to \bot) \to (\psi \to \bot)$.

  1. $\Gamma \vdash (\varphi \to \bot) \to (\psi \to \bot)$
  2. $\Gamma \vdash ((\varphi \to \bot) \to (\psi \to \bot)) \to (((\varphi \to \bot) \to \psi) \to ((\varphi \to \bot) \to \bot))$ (Ax 2)
  3. $\Gamma \vdash ((\varphi \to \bot) \to \psi) \to ((\varphi \to \bot) \to \bot)$ (MP 1, 2)
  4. $\Gamma \vdash (\varphi \to \bot) \to \psi$
  5. $\Gamma \vdash (\varphi \to \bot) \to \bot$ (MP 3, 4)
  6. $\Gamma \vdash ((\varphi \to \bot) \to \bot) \to \varphi$ (Ax 3)
  7. $\Gamma \vdash \varphi$ (MP 5, 6)

  □

- (b) Prove Lemma 2

  Proof. By definition, $\Gamma \cup \{\varphi\} \vdash \psi$ and $\Gamma \cup \{\varphi\} \vdash (\psi \to \bot)$ for some ψ. By the Deduction:

  1. $\Gamma \vdash \varphi \to \psi$
  2. $\Gamma \vdash \varphi \to (\psi \to \bot)$
  3. $\Gamma \vdash (\varphi \to (\psi \to \bot)) \to ((\varphi \to \psi) \to (\varphi \to \bot))$ (Ax 2)
  4. $\Gamma \vdash (\varphi \to \psi) \to (\varphi \to \bot)$ (MP 2, 3)
  5. $\Gamma \vdash \varphi \to \bot$ (MP 1, 4)

  □

- (c) Let Γ be a set of formulae and φ be a formula. Show that, if Γ is consistent, then at least one of $\Gamma \cup \{\varphi\}$ or $\Gamma \cup \{\varphi \to \bot\}$ is consistent

  Proof. Suppos otherwise, Γ is consistent, $\Gamma \cup \{\varphi\}$ and $\Gamma \cup \{\varphi \to \bot\}$ are inconsistent. Since $\Gamma \cup \{\varphi\}$ is inconsistent, by Lemma 2

  $$\Gamma \vdash \varphi \to \bot .$$

  Since $\Gamma \cup \{\varphi \to \bot\}$ is inconsistent, by Lemma 1 we have

  $$\Gamma \vdash \varphi.$$

  $$\Gamma \vdash \bot .$$

  By explosion, pick an arbitrary formula $\psi, \Gamma \vdash \bot \vdash \psi, \Gamma \vdash \bot \vdash \psi \to \bot, \Gamma$ is inconsistent, contradiction.
  □

- (d) Let Γ, Γ' be sets of formulae such that $\Gamma \subseteq \Gamma'$. If Γ' is consistent, then so is Γ

  Proof. Suppose othrewise, Γ is inconsistent,

  $$\Gamma \vdash \bot .$$

  $\Gamma \subseteq \Gamma'$, by monotonicity of derivability we have

  $$\Gamma' \vdash \bot .$$

  Contradiction. □

- (e)

  Proof. Γ is inconsistent, $\exists \varphi, \Gamma \vdash \varphi, \Gamma \vdash (\varphi \to \bot)$ there exists a finite proof for both φ and $\varphi \to \bot$. Consider

  - $\pi_1 = \langle \phi_1, \phi_2, \ldots, \phi_n \rangle, \phi_n = \varphi$
  - $\pi_2 = \langle \psi_1, \psi_2, \ldots, \psi_m \rangle, \psi_m = \varphi \to \bot$

  We construct Γ' as such: $\forall i < n, j < m$, if $\phi_i \in \Gamma, \phi_i \in \Gamma'$; if $\psi_j \in \Gamma, \psi_j \in \Gamma'$ Therefore $\forall \eta \in \Gamma', \eta \in \Gamma, \Gamma' \subseteq \Gamma$; $|\Gamma'| \leq n+m, \Gamma'$ is a finite subset of Γ. Since pi_1 is a proof for ϕ, $\forall \phi \in \pi_1, \phi \in \Gamma \implies \phi \in \Gamma'$; We can easily show that same proof can be constructed from Γ' by induction. Same case for $\Gamma' \vdash \varphi \to \bot$. Therefore Γ' is inconsistent. □

Definition 9.60.

- $\Delta_0 = \{\varphi \to \bot\}$
- $\Delta_{i+1} = \begin{cases} \Delta_i \cup \{\varphi_i\} \text{ if } \Delta_i \cup \{\varphi_i\} \text{ is consistent} \\ \Delta_i \cup \{\varphi_i \to \bot\} \text{ otherwise} \end{cases}$

$$\Delta = \bigcup_{i \geq 0} \Delta_i$$

Lemma 9.61. *Γ is consistent $\implies$ $\Gamma \cup \{\varphi\}$ or $\Gamma \cup \{\varphi \to \bot\}$ is consistent.*

Question 7

- (a) Prove, using induction, that Δ_i is consistent, for every i

  Proof. **Base case:** Δ_0

  Suppose otherwise, $\Delta_0 = \{\varphi \to \bot\}$ is inconsistent. Then by Lemma 1,

  $$\vdash \varphi.$$

  Contradiction.

  Inductive case: Suppose $\forall j \leq i, \Delta_j$ is consistent

  By lemma above, Δ_i is consistent $\implies$ $\Delta_i \cup \{\varphi_i\}$ is consistent or $\Delta_i \cup \{\varphi_i \to \bot\}$ is consistent $\implies$ Δ_{i+1} is consistent □

- (b) Prove that Δ is consistent

  Proof. Suppose otherwise, Δ is inconsistent, by Lemma 3, $\exists \Delta' \subseteq_{fin} \Delta, \Delta'$ is inconsistent. Since Δ' is finite, let the highest ranked formula in Δ' be $n, \forall \varphi \in \Delta', \varphi \in \Delta_n$. From b), Δ_n is consistent, by 6d), Δ' is consistent, contradiction. □

- (c) Prove that Δ is negation-complete, i.e., for every formula ψ, we have $(\psi \to \bot) \in \Delta$ iff $(\psi \notin \Delta)$

  Proof. $\implies$

  Let $\psi = \varphi_{n+1}, \Delta_n$ is consistent. $\Delta_{n+1} = \Delta_n \cup \{\psi\}$ if $\Delta_n \cup \{\psi\}$ is consistnet. Suppose $\psi \to \bot \in \Delta$, $\Delta \vdash \psi, \Delta \vdash \psi \to \bot \implies \Delta$ is inconsitent, contradiction. Therefore $\psi \in \Delta \implies \psi \to \bot \notin \Delta$

  $\impliedby$

  Suppose otherwise, $\psi \to \bot \in \Delta, \psi \in \Delta, \Delta \vdash \psi, \Delta \vdash \psi \to \bot \implies \Delta$ is inconsistent, contradiction. □

- (d) Use the fact that Δ is negation-complete to show that for every formula ψ, $\psi \in \Delta$ iff $\Delta \vdash \psi$

Proof. $\implies$

trivial by definition

$\impliedby$

Consider $\Delta \vdash \psi$. Suppose $\psi \notin \Delta, \psi \to \bot \in \Delta, \Delta \vdash \psi \to \bot$. Since $\Delta \vdash \psi$, by MP, $\Delta \vdash \bot$, Δ is inconsistent, contradiction. Therefore $\psi \in \Delta$. □

Question 8

- (a) Show that for any formula ψ, $v \models \psi$ iff $\psi \in \Delta$

Proof. **Base case:**

1. $\psi \in P$: by definition, $v(p) = true \iff p \in \Delta$
2. $\psi = \bot$: always $v(\bot) = false, \bot \notin \Delta \implies v \models \bot \iff \bot \in \Delta$ vacuously

Inductive case: $\psi = a \to b$

Suppose

- $v \models a \iff a \in \Delta$
- $v \models b \iff b \in \Delta$

Case $\implies$

Suppose $v \models a \to b$, $a \to b \notin \Delta, (a \to b) \to \bot \in \Delta$, $\Delta \vdash (a \to b) \to \bot$.
Now we show that $b \notin \Delta$

- Suppose otherwise, $b \in \Delta, \Delta \vdash b$, by ax 1, $\Delta \vdash b \to (a \to b) \implies \Delta \vdash a \to b$ by MP, contradiction.

Now we show that $a \in \Delta$

- Suppose otherwise, $a \notin \Delta \implies a \to \bot \in \Delta$, from

$$\vdash (\varphi \to \bot) \to (\varphi \to \rho)$$

$\Delta \vdash (a \to \bot) \to (a \to b)$. By MP, $\Delta \vdash a \to b$, contradiction.

Now with $b \notin \Delta$ and $a \in \Delta$, by IH, $v \models a$ and $v \not\models b \implies v \not\models a \to b$, contradiction.

Case $\impliedby$

Suppose otherwise, $a \to b \in \Delta$, $v \models a$ and $v \not\models b$. By IH, $a \in \Delta$ and $b \notin \Delta$, $b \to \bot \in \Delta \implies \Delta \vdash a, \Delta \vdash a \to b$, by MP, $\Delta \vdash b \implies b \in \Delta$, contradiction. □

- (b) Show that $v \models (\varphi \to \bot)$ and thus $\not\models \varphi$

Proof. $\{\varphi \to \bot\} \in \Delta \implies v \models (\varphi \to \bot) \implies v \not\models \varphi \implies \not\models \varphi$. □

Question 9

Prove Theorem 3 using Theorem 4, the Compactness theorem and the Deduction theorem

Proof. $\Gamma \models \varphi \implies \Gamma \cup \{\varphi \to \bot\}$ is unsat, so by Compactness $\exists \Gamma_0 = \{\psi_1, \dots, \psi_n\} \subseteq_{\text{fin}} \Gamma$, such that $\Gamma_0 \cup \{\varphi \to \bot\}$ is unsat,

$$\{\psi_1, \dots, \psi_n, (\varphi \to \bot)\} \text{ is unsat } \implies (\psi_1 \wedge \dots \wedge \psi_n \wedge (\varphi \to \bot)) \models \bot$$

$$\models \psi_1 \to (\psi_2 \to \dots (\psi_n \to ((\varphi \to \bot) \to \bot)) \dots)$$

(https://en.wikipedia.org/wiki/Exportation_(logic)) by Theorem 4

$$\emptyset \vdash \psi_1 \to (\psi_2 \to \dots (\psi_n \to ((\varphi \to \bot) \to \bot)) \dots)$$

Apply deduction thm

$$\{\psi_1\} \vdash (\psi_2 \to \dots (\psi_n \to ((\varphi \to \bot) \to \bot)) \dots)$$

until we get

$$\Gamma_0 \vdash (\varphi \to \bot) \to \bot.$$

Using ax 3:

$$\vdash ((\varphi \to \bot) \to \bot) \to \varphi,$$

By MP

$$\Gamma_0 \vdash \varphi.$$

Since $\Gamma_0 \subseteq \Gamma$, by monotonicity,

$$\Gamma \vdash \varphi.$$

□

Theorem 5 (Soundness of Resolution Proof System)

Let Γ be a set of clauses (over propositions P). If there is a resolution proof of Γ, then Γ is unsatisfiable.

Question 10

Prove Theorem 5, *Hint: Consider a resolution proof* $\pi = C_1, C_2, \dots, C_n$ *of* Γ. *Use induction to establish that for every* $1 \leq i \leq n$ *if a valuation* v *is such that* $v \models \Gamma$, *then* $v \models C_i$.

Proof. Let $\pi = C_1, C_2, \dots, C_n$ be a resolution proof of Γ, where $C_n = \emptyset$.
We prove by induction on i that for every $1 \leq i \leq n$, $v \models \Gamma, v \models C_i$.
$i \in \{1, \dots, n\}$ and suppose $v \models \Gamma$.

1. $C_i \in \Gamma$, $v \models C_i$.
2. C_i is a resolvent of two earlier clauses C_j and C_k wrt some proposition p, where

$$C_j = C \uplus \{p\}, \qquad C_k = D \uplus \{\neg p\},$$

and $C_i = C \cup D$. By the IH, $j, k < i, v \models C_j$ and $v \models C_k$ We show that $v \models C_i$. Suppose, $v \not\models C_i = C \cup D$. Then v satisfies neither any literal in C nor any literal in D. Since $v \models C_j = C \uplus \{p\}$ and no literal in C is satisfied by v, it must be that $v \models p$, i.e. $v(p) =$ true Since $v \models C_k = D \uplus \{\neg p\}$ and no literal in D is satisfied by v, it must be that $v \models \neg p$, i.e. $v(p) =$ false, Contradicton

□

Theorem 6 (Completeness of Resolution Proof System)

Let Γ be a set of clauses (over propositions P). If Γ is unsatisfiable, then there is a resolution proof of Γ

Question 11

Let Γ be an unsatisfiable set of clauses Show that, if Γ is unsatisfiable, then there is a finite set $\Gamma_0 \subseteq_{fin} \Gamma$ such that Γ_0 is unsatisfiable.

Proof. Suppose Γ is unsat,by Compactness, $\exists \Gamma' \subseteq_{fin} \Gamma, \Gamma'$ is unsat (?) □

Question 12

Let Γ_1, Γ_2 be sets of clauses such that $\Gamma_1 \subseteq \Gamma_2$. Show that, if there is a resolution proof of Γ_1, then there is also a resolution proof of Γ_2

Proof. Let $\pi = C_1, C_2, \ldots, C_n$ be a resolution proof of Γ_1
$\forall 1 \leq i \leq n$, π is a resolution proof of Γ_1, either:

1. $C_i \in \Gamma_1$, or
2. $\exists j, k < i$, C_i is a resolvent of C_j and C_k

If $C_i \in \Gamma_1, C_i \in \Gamma_2$ Otherwise, this fact is independent of the choice of the initial clause set, so the same derivation is valid in a proof from Γ_2. □

Question 13

Next, we constructed a sequence of sets as follows. Here, we assume that the set of propositions occurring in Γ_0 can be enumerated as $p_1, p_2, \ldots, p_k$.

$$\Delta_0 = \Gamma_0$$

$$\Delta_{i+1} = \Delta_i^{\emptyset} \cup \{\, C \cup D \mid C \uplus \{p_{i+1}\} \in \Delta_i^{+},\ D \uplus \{\neg p_{i+1}\} \in \Delta_i^{-} \,\}$$

where

- $\Delta_i^{\emptyset}$ is the set of clauses in Δ_i that contain neither the literal p_{i+1} nor $\neg p_{i+1}$,
- Δ_i^{+} is the set of clauses in Δ_i that contain the literal p_{i+1} but not $\neg p_{i+1}$,
- Δ_i^{-} is the set of clauses in Δ_i that contain the literal $\neg p_{i+1}$ but not p_{i+1}.

Inductively show that Δ_i is unsat for every $0 \leq i \leq k$. Also show that $\Delta_i \neq \emptyset$ for every $0 \leq i \leq k$.

Proof. **Base case:**

$i = 0$. By construction, $\Delta_0 = \Gamma_0$. From 11, Γ_0 is a finite unsat subset of Γ

Inductive step:

Suppose Δ_i is unsat and $\Delta_i \neq \emptyset$. We show Δ_{i+1} is unsat and nonempty. Suppose that $\exists v, v \models \Delta_{i+1}$, consider $v(p_{i+1})$

- If $v(p_{i+1}) = \text{true}$, then every clause in Δ_i^{+} is sat by v due to p_{i+1}, and $\Delta_i^{\emptyset} \subseteq \Delta_{i+1}$ thus sat by v.
  For every resolvent $C \cup D \in \Delta_{i+1}$ with

  $$C \uplus \{p_{i+1}\} \in \Delta_i^{+}, \quad D \uplus \{\neg p_{i+1}\} \in \Delta_i^{-},$$

  $v \models C \cup D$. Since $v(p_{i+1}) = \text{true}$, the clause $D \uplus \{\neg p_{i+1}\}$ must be satisfied by some literal in D, hence $v \models D$. Thus v satisfies every clause in Δ_i^{-}. Contradiction
- case $v(p_{i+1}) = \text{false}$: similar

□

Question 14

Inductively show that Δ_i does not contain the literals $\{p_1, \neg p_1, \ldots, p_i, \neg p_i\}$ for every $1 \leq i \leq k$

Proof. We prove by induction on i that Δ_i does not contain any of the literals

$$\{p_1, \neg p_1, \ldots, p_i, \neg p_i\}$$

for every $1 \leq i \leq k$.

Base case:

$i = 1$.

By definition,

$$\Delta_1 = \Delta_0^{\emptyset} \cup \{\, C \cup D \mid C \uplus \{p_1\} \in \Delta_0^{+},\ D \uplus \{\neg p_1\} \in \Delta_0^{-} \,\}.$$

Every clause in $\Delta_0^{\emptyset}$ contains neither p_1 nor $\neg p_1$, and every resolvent $C \cup D$ is obtained by removing p_1 and $\neg p_1$. Hence no clause in Δ_1 contains p_1 or $\neg p_1$.

Inductive step:

□

Question 15

Finish the proof of Theorem 6 by explicitly constructing a resolution proof for Γ and establishing its correctness

Proof. Let Γ be unsat. By 11, $\exists \Gamma_0 \subseteq_{fin} \Gamma$ such that Γ_0 is unsat. Let $p_1, \ldots, p_k$ be all propositions occured in Γ_0, and $\Delta_0, \Delta_1, \ldots, \Delta_k$ as in the construction. Δ_k is unsat, and by 14, Δ_k does not contain $p_1, \neg p_1, \ldots, p_k, \neg p_k$. therefore every clause in Δ_k contains no literal,thus every clause in Δ_k is $\emptyset$. Since $\Delta_k \neq \emptyset$, $\emptyset \in \Delta_k$

Base case:

$i = 0$, $\Delta_0 = \Gamma_0$

Inductive step:

Suppose every clause in Δ_i has a finite resolution proof from Γ_0. Let $E \in \Delta_{i+1}$. By definition ,

1. $E \in \Delta_i^{\emptyset}$, hence $E \in \Delta_i$, so by IH E has a finite resolution proof from Γ_0; or
2. $E = C \cup D$, where
$$C \uplus \{p_{i+1}\} \in \Delta_i^{+} \qquad \text{and} \qquad D \uplus \{\neg p_{i+1}\} \in \Delta_i^{-}.$$
Then E is the resolvent of these 2
$$C \uplus \{p_{i+1}\} \quad \text{and} \quad D \uplus \{\neg p_{i+1}\}.$$
By IH, both clauses have a finite resolution proof from Γ_0, thus the resolution proof for E is also finite

$\Gamma_0 \subseteq \Gamma$, by 12 the same proof is also a resolution proof fo Γ.
Therefore Γ is unsat, then there is a resolution proof for Γ. □

We will prove the contrapositive of compactness theorem for the next 2 qn.

Question 16

Assume that the Hilbert-style Proof System is sound and complete for propositional logic, Use these facts to establish that the Compactness Theorem holds true for propositional logic

Proof. Γ is unsat $\implies$ $\Gamma \models \bot$, so by completeness $\Gamma \vdash \bot$ since proofs are finite, only finitely many assumptions from Γ appear in the proof, yielding a finite $\Gamma_0 \subseteq_{\text{fin}} \Gamma$ with $\Gamma_0 \vdash \bot$, and by soundness $\Gamma_0 \models \bot$, hence Γ_0 is unsat □

Question 17

Assume that the Resolution Proof System is sound and complete for propositional logic. Use these facts to establish that the Compactness Theorem holds true for propositional logic

Proof. If Γ is an unsatidfiable set of clauses, by completeness there is a finite resolution proof for Γ, and since this proof uses only finitely many clauses from Γ, there is a finite subset $\Gamma_0 \subseteq_{\text{fin}} \Gamma$ from which the same proof is a resolution proof. By soundness, Γ_0 is unsat $\square$